



Research paper

HOIDS: Concept drift aware hybrid online intrusion detection system

Alessandra De Paola^{a,c}, Salvatore Drago^b, Pierluca Ferraro^{a,c}, Giuseppe Lo Re^{a,c}^a Department of Engineering, University of Palermo, Palermo, Italy^b IMT School for Advanced Studies Lucca, Lucca, Italy^c Cybersecurity National Lab, CINI - Consorzio Interuniversitario Nazionale per l'Informatica, Italy

ARTICLE INFO

Keywords:

Network security
Threats detection
Machine learning
Intrusion Detection System
Online learning
Concept drift

ABSTRACT

Most Intrusion Detection Systems (IDS) based on machine learning operate under the assumption of a closed environment, where the statistical properties of the data remain constant over time. However, in real-world scenarios, this assumption is rarely valid. The data generation process is often non-stationary, leading to shifts in the statistical distribution of the data over time. This phenomenon, known as concept drift, causes significant performance degradation in static systems, which often require manual retraining by human operators, leaving networks resources vulnerable in the meantime. To address these challenges, we propose an Intrusion Detection System that adopts an online learning and drift-aware approach, incorporating concept drift detection and adaptation algorithms to handle evolving environments. One of the main challenges posed by concept drift is the need for frequent retraining, which increases the cost of real-time data labeling. As a solution to this problem, the system we designed employs a hybrid approach, combining supervised and unsupervised algorithms. The higher performance of supervised models is leveraged for detection, while unsupervised techniques are used to filter and minimize the amount of data requiring human review, thus reducing labeling costs. The system has been extensively evaluated on real-world datasets, demonstrating its capability of outperforming other state-of-the-art methods, achieving an F1-score of 0.961 while reducing the labeling cost to 0.92% of the dataset, compared to 100% required by systems with comparable detection performance. Thanks to its adaptive handling of concept drift and efficient filtering mechanisms, the system proved to be an effective and practical solution for real-world intrusion detection scenarios.

1. Introduction

In recent years, due to the proliferation of attacks on computer systems, cybersecurity detection mechanisms deserve more and more interest. In such area, Intrusion Detection Systems (IDS) have emerged as some of the most thoroughly researched and implemented systems, playing a crucial role in identifying and mitigating potential threats.

Machine learning techniques are today widely applied to develop autonomous and intelligent IDS capable of analyzing network traffic and detecting potentially malicious activities. However, despite significant progress in the development of machine learning-based IDSs, several critical challenges remain underexplored, and addressing them is essential for a robust and effective deployment of such systems.

A common limitation of these systems is their static nature. Most of the machine learning based IDSs in the literature are based on the assumption of a *closed environment*, that means the statistical properties of the data-generating processes remain constant over time. In many real-world scenarios, however, the assumption of a closed environment

does not hold. Once an IDS is deployed, there is no guarantee that the network traffic analyzed will follow the same statistical distribution as the data observed during the training phase. This scenario is known in the literature as *open environment*.

For instance, let us consider an IDS designed to analyze the network traffic generated and received by a university campus or private company. Such a system might be trained on a comprehensive dataset containing benign and malicious traffic records, labeled by domain experts over time during regular activities. However, with the experience of the Covid-19 pandemic, we have learned that a drastic change in traffic patterns can occur, in that case due to the shift to exclusively online activities, involving platforms and services never used before.

The previous scenario is an example of a non-stationary data generation process, which can cause shifts in the statistical distribution of data over time. This phenomenon, known as *concept drift*, invalidates previous training and introduces new errors, leading to severe performance degradation in static systems. Such systems often require manual

* Corresponding author.

E-mail addresses: alessandra.depaola@unipa.it (A. De Paola), salvatore.drago@imtlucca.it (S. Drago), pierluca.ferraro@unipa.it (P. Ferraro), giuseppe.lore@unipa.it (G. Lo Re).

<https://doi.org/10.1016/j.jnca.2026.104556>

Received 8 January 2026; Received in revised form 12 June 2026; Accepted 17 July 2026

Available online 23 July 2026

1084-8045/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

retraining by human operators, leaving networks resources vulnerable in the meantime.

The most promising strategy for handling open environments is to use online or incremental learning techniques that can manage data streams while incorporating concept drift detection and adaptation algorithms. However, several aspects of online learning under concept drift are still underexplored, limiting the general application of this approach in the field of intrusion detection systems.

In many works in the literature, both the concept drift detection and the adaptation phases assume that, after a certain time interval, the ground truth labels for the evaluated data become available. During the concept drift detection phase, the system compares the ground truth labels with the predicted labels to identify drift through performance degradation. In the adaptation phase, supervised techniques, which have been proven to perform well even on high-dimensional datasets such as real network traffic in stationary environments, use the ground truth data to retrain or update their model when drift is detected.

This additional labeling phase requires real-time human intervention, which is often impractical in terms of both timing and volume, especially in domains like online learning under concept drift for intrusion detection systems. In the literature, this challenge is referred to as the *cost of labeling*.

To address these limitations, we propose HOIDS, an Intrusion Detection System that adopts an online learning and drift-aware approach, designed to operate under evolving (open) environments. This approach detects drifts over time and adapts by retraining the underlying machine learning model only when necessary. The system adopts a multi-level architecture that integrates concept drift detection and adaptation modules with both supervised and unsupervised techniques in a hybrid approach. This design leverages the high performance of supervised models while simultaneously reducing labeling costs by using unsupervised techniques, such as online active learning, to filter the records that require review by domain experts.

To this end, advanced unsupervised learning techniques, such as autoencoders are adopted, since they are particularly effective in distinguishing benign from uncertain traffic, thus reducing the need for manual labeling. Furthermore, a novel multi-dimensional approach to concept drift detection, called MPCAWIN (Multidimensional PCA-based Windowing), is introduced. This method enhances the system's ability to adapt to different types of drift while maintaining robustness against noise. MPCAWIN, which is based on the statistical distribution of input data features, allows for more accurate drift detection, further minimizing the cost of labeling during the detection phase.

The system's performance is evaluated using real network datasets that are considered benchmarks for IDS, both in intra-dataset and inter-dataset scenarios. This evaluation demonstrates the system's effectiveness under both closed and open environment assumptions, and highlights the impact of concept drift. The system is compared to other state-of-the-art methods, highlighting the strengths and weaknesses of different approaches.

Unlike existing drift-aware IDS approaches which rely on error-rate-based detectors requiring continuous access to ground-truth labels to monitor classifier performance, HOIDS eliminates this dependency entirely during the detection phase. Furthermore, error-rate-based detectors are sensitive to noise and can trigger unnecessary retraining phases, wasting both computational and labeling resources. HOIDS instead retrains only upon genuine drift detection, avoiding unnecessary adaptation steps while maintaining comparable or superior detection performance, as demonstrated in Section 5. Finally, while existing systems assume that the full retraining window is annotated by domain experts at each adaptation step, HOIDS reduces this cost to a small fraction of the data through a novel active learning module. The combination of these properties constitutes an end-to-end design that other drift-aware IDS approaches in the literature do not achieve.

The key contributions of this paper are listed below:

- A hybrid online Intrusion Detection System that eliminates ground-truth labeling requirements during the drift detection phase and minimizes labeling costs during the adaptation phase, while maintaining high detection performance in the presence of significant concept drifts.
- An online active learning (AL) based on an AutoEncoder that simultaneously auto-labels the majority of benign records in the retraining window without expert intervention, and identifies the uncertain records to be submitted for manual review. An optimized iterative version (IAE) further refines this process to enhance performance.
- An in-depth analysis of the risk of false negatives and false positives propagating into the retraining set during the adaptation phase as the labeling cost varies, which shows the importance of the robustness of the proposed AL module and its IAE version, as well as their ability to identify and automatically label benign records while avoiding misclassifications.
- An unsupervised multi-dimensional concept drift detection method (MPCAWIN) based on the statistical distribution of input features, which requires no ground-truth labels during the detection phase and is therefore more practical than error-rate-based detectors in scenarios where timely annotation is infeasible. MPCAWIN also enhances robustness to noise and handles multiple types of drift. The code of MPCAWIN is publicly available on GitHub.¹
- An extensive experimental validation on multiple real-world datasets from different network domains containing concept drift, which assesses the impact of drift on static systems and demonstrates the generalizability of HOIDS. Such evaluation also confirms its competitive performance against state-of-the-art online systems representing the main concept drift adaptation strategies, while reducing labeling costs.
- An ablation study on the individual components of HOIDS, proving that its modular hybrid architecture enables a flexible and tunable trade-off between labeling cost and computational overhead across different operational scenarios.

The remainder of the paper is structured as follows. Section 2 discusses related work. Section 3 provides background on concept drift, including its types and causes, as well as detection and adaptation strategies. Section 4 describes the details of the system we propose. The experimental evaluation is presented in Section 5. Finally, Section 6 draws our conclusions and future work.

2. Related work

The exponential growth in speed, complexity, and volume of network traffic generated by modern computer networks has promoted the adoption of flow-based Intrusion Detection Systems (IDS) (Abuadlla et al., 2014; Vo et al., 2023), as opposed to traditional payload-based IDS which rely on deep packet inspection. Flow-based IDS can be broadly classified into two categories based on how traffic is analyzed (Liao et al., 2013): *signature-based* and *behavior-based* detection, also known as *misuse-based* and *anomaly-based*, respectively. Signature-based detection compares the signatures of known attacks with the analyzed traffic, making it effective against known threats but inadequate for detecting unknown or zero-day attacks. Behavior-based IDSs address this limitation by analyzing network log files to identify deviations from normal patterns. These systems can detect previously unknown (zero-day) attacks by recognizing disruptions in expected behavior.

However, defining what constitutes normal behavior is challenging. As a result, anomaly-based techniques (Shana et al., 2025) typically

¹ <https://github.com/ndslab-group/MPCAWIN>.

show higher false positive rates compared to other methods. Recent research has focused on the use of statistical techniques and various machine learning models to define normal and anomalous behavior from realistic network traffic datasets, enabling intrusion detection systems to automatically identify events likely indicative of an attack.

Many studies in the literature propose machine learning-based IDS that exploit different techniques. Unsupervised and semi-supervised (Perumal et al., 2025; Li et al., 2020) approaches have the advantage of not requiring labeled data to identify potentially malicious traffic as anomalous, in contrast to training datasets containing benign network traffic captured during legitimate user activity. However, unsupervised and semi-supervised models suffer significantly from the issue of high dimensionality, a challenge common to all major real-world network datasets currently used as benchmarks for Network Intrusion Detection Systems (NIDS). In contrast, supervised models trained on large, manually labeled network datasets by domain experts, such as Deep Neural Networks (DNN) (Wahab, 2022), decision trees, and ensemble models, have demonstrated strong performance, even on multi-class tasks. These models can accurately distinguish between benign and malicious traffic and further identify the specific type of attack responsible for the malicious activity.

One of the most promising machine learning approaches is based on ensemble techniques using decision trees to combine the strengths of multiple models. A notable example is the recent LCCDE framework (Yang et al., 2022), an ensemble IDS designed to protect Internet of Vehicles (IoV) systems from cyber threats. This framework employs three advanced ML algorithms (XGBoost, LightGBM, and CatBoost) using leader models for each attack class and their prediction confidence to make accurate decisions about different types of cyber-attacks. The system achieves excellent results across all common evaluation metrics.

The main limitation of this and other state-of-the-art Network Intrusion Detection Systems (NIDS) is their reliance on the closed environment assumption, where the statistical characteristics of the data-generating processes are assumed to be stable over time. However, this assumption often fails in real-world scenarios.

In dynamic environments, such as the current Internet, data streams are often influenced by non-stationary phenomena. This leads to what is known as concept drift (Gama et al., 2014; Lu et al., 2018; Hurtado et al., 2023) (Section 3), where the distribution of input or output data shifts unpredictably over time, creating significant challenges for static detection methods. Indeed, static systems experience abrupt performance degradation as a result of concept drift. This issue is also highlighted in D'hooge et al. (2020) and further demonstrated in this work (Section 5) by comparing intra-dataset performance, where systems operate under a closed environment assumption, with inter-dataset performance, where systems must handle dynamic (open) environments. This performance degradation occurs even when datasets are collected in a similar manner, share the same features, and exhibit similar characteristics.

One of the most effective strategies for managing open environments is to employ online or incremental learning techniques, which can handle data streams while integrating concept drift detection and adaptation algorithms. Two recent and noteworthy online systems which explicitly address concept drift are *AutoML* (Celik and Vanschoren, 2021) and *AXGB* (Montiel et al., 2020).

In *AutoML*, the authors examine the impact of concept drift on the performance of Automated Machine Learning (AutoML) methods and discuss adaptation strategies that can enhance their robustness towards changes in the underlying data. *AXGB* (Adaptive XGBoost) is an adaptation of XGBoost for classifying evolving data streams affected by concept drift. The authors propose an incremental training algorithm for ensemble learners, along with various replacement policies for learners in the ensemble. They also explore a concept drift detection phase using a sliding window of variable size, decreasing the window size when drift is detected and increasing it to the maximum when drift is not observed.

The main limitation of these approaches is their dependence on supervised algorithms and supervised error-based concept drift detection methods (Section 3), which, like most online learning systems dealing with concept drift, assume that ground truth labels are available immediately after classification or prediction. This assumption ignores the significant issue of labeling costs, which is impractical in online contexts due to unrealistic expectations for real-time labeling by domain experts, especially in domains like intrusion detection systems.

One possible strategy is to use online supervised models under the assumption of *limited labeling*. In this context, an online IDS may only rely on a subset of the available data to reduce the labeling cost during the adaptation phase. These subsets can be selected either through random sampling or by applying *active learning techniques* (Tharwat and Schenck, 2023), which aim to identify the most informative instances for retraining.

The work described here proposes an online approach designed to reduce labeling costs by minimizing the amount of labeled data required, thus decreasing the need for expert intervention.

3. Background on concept drift management

This section provides an overview of concept drift and its implications for online machine learning models, describing the types and causes of drift, as well as discussing various detection and adaptation strategies necessary to maintain performance in dynamic environments. *Concept drift* (Lu et al., 2018) is defined as a shift in the distribution of input or output data over time. Formally, a concept drift occurs when:

$$\exists t : P_t(X, y) \neq P_{t+\delta}(X, y), \quad (1)$$

where X is a set of features and y is a set of corresponding labels, while $P(X, y)$ represents the joint probability of X and y . The joint probability $P_t(X, y)$ can be decomposed as $P_t(X, y) = P_t(X) \times P_t(y|X)$. This distinction makes it possible to identify different causes of concept drift according to the component that has changed, as shown in Fig. 1.

Specifically, when only the distribution of the input data changes, $P_t(X) \neq P_{t+\delta}(X)$, without shifts in the predictions $P_t(y|X) = P_{t+\delta}(y|X)$, this is known as *virtual drift* (Ramírez-Gallego et al., 2017) or *feature space drift*, as shown in Fig. 1-a. Conversely, *actual drift* or *decision boundary drift* occurs when the distribution of input data remains unchanged, $P_t(X) = P_{t+\delta}(X)$, but the predictions change, i.e., $P_t(y|X) \neq P_{t+\delta}(y|X)$, causing the decision boundary to be altered, as shown in Fig. 1-b. Fig. 1-c shows the third type of drift, which occurs when both the distribution of input data and predictions change together, i.e., $P_t(X) \neq P_{t+\delta}(X)$ and $P_t(y|X) \neq P_{t+\delta}(y|X)$. This scenario is the most realistic in real contexts such as the Internet and network traffic.

Generally, four types of possible drifts are distinguished, as shown in Fig. 2:

- *sudden*: it occurs, for example, when there is an unexpected event, such as the transition to online-only activities with new platform and services;
- *gradual*: this occurs when a new concept in its initial phase alternates with the previous one before stabilizing and finally prevailing, as during the testing and subsequent implementation of new hardware or software services.
- *incremental*: involves a sequence of intermediate concepts that follow one another, for example to reflect changing user preferences over time or the slow migration to a new network service, such as the introduction of optional two-factor authentication that later becomes mandatory;
- *recurring*: it occurs when old concepts repeat themselves in cycles, such as according to seasonal changes or periodic events, such as returning to normal after an unusual period.

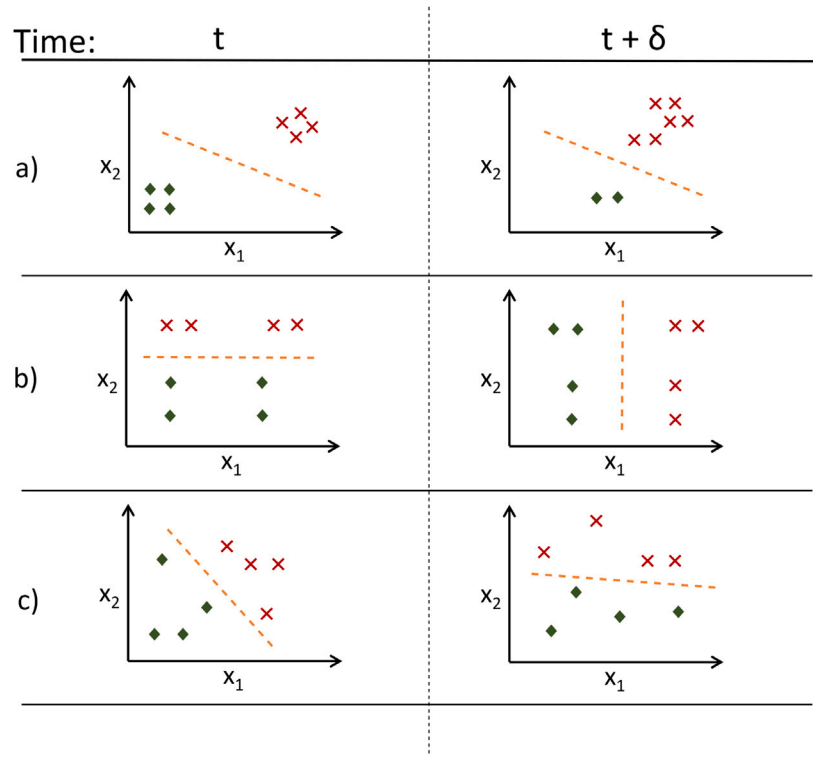


Fig. 1. Sources of concept drift.

The authors of [Hinder et al. \(2022\)](#) extend the definition in Eq. (1) by explicitly modeling the temporal structure of the data-generating process through a time model T , rather than a simple index set. The framework assumes a probability distribution P_T over T , characterizing the likelihood of observing a data point at time t , together with a collection of conditional distributions D_t for all $t \in T$, even though in practice only a finite number of time points is observed.

The combination of P_T and D_t forms a *distribution process*, on which the authors of [Hinder et al. \(2024\)](#) build to define concept drift as a property of the data-generating process itself, rather than of a specific drawn sample. Formally, a distribution process D_t exhibits concept drift if, with nonzero probability, a sample $X_1, X_2, \dots$ drawn from it contains indices i and j such that

$$P_{X_i} = D_{T_i} \neq D_{T_j} = P_{X_j}. \quad (2)$$

In any case, it is essential to provide strategies to detect the concept drift and trigger a quick and efficient adaptation phase to re-adapt the machine learning model.

The most common approach, shown in [Fig. 3](#), handles a continuous stream of data and has two main phases: Concept Drift Detection (CDD) and Concept Drift Adaptation (CDA). When the Concept Drift Detection identifies a drift, the Concept Drift Adaptation responds by updating or replacing the machine learning model and initiating a new training phase. This dynamic process ensures that the system adapts to changes over time, maintaining its performance and accuracy.

The *Concept Drift Detection* generally consists of four sub-phases ([Hinder et al., 2024; Lu et al., 2018](#)):

1. *Data Retrieval*, which concerns data flow management. Sliding windows and batch techniques are often used to compare the statistical distribution of past data with that of new data;
2. *Data Modeling*, which aims to identify and extract key characteristics of data that are critical, i.e., those that have the greatest impact on system performance in the event of concept drift;
3. *Statistic Test Calculation*, which consists of computing a measure of dissimilarity (or distance) between the two data distributions identified in step 1;

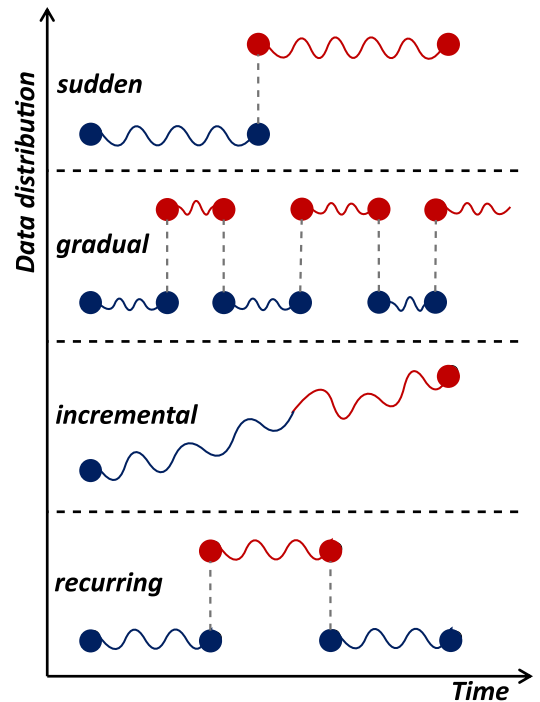


Fig. 2. Types of concept drift.

4. *Hypothesis Test*, used to evaluate the probability that the observed dissimilarity is caused by concept drift instead of simple data noise. This step is crucial to avoid misinterpreting data noise as concept drift. However, in many studies, this step is omitted ([Ding and Fei, 2013](#)), and the score from step 3 is directly compared to a static threshold. The most commonly used

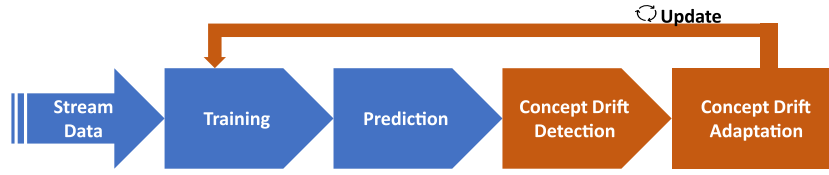


Fig. 3. Common workflow for handling concept drift.

hypothesis test involves estimating the statistical distribution of the dissimilarity score calculated in step 3.

Two macro-categories of Concept Drift Detection algorithms can be distinguished. The first macrocategory is based on model error rate (error rate-based drift detection); here the Data Modeling phase coincides with the model prediction phase, and the Statistic Test Calculation phase compares the model predictions with the ground truth data labels. The Hypothesis Test is implemented through widely used algorithms in the literature, such as ADWIN (Bifet and Gavalda, 2007), EDDM (Baena-Garcia et al., 2006), HDDM_A (Frias-Blanco et al., 2014), HDDM_W (Frias-Blanco et al., 2014), Page-Hinkley (Page, 1954) and KSWIN (Raab et al., 2020). The above algorithms were originally designed for use with mono-dimensional data, as in the case of error rate. In such scenarios, the degradation in system performance is attributed to concept drift, which subsequently triggers an adaptation phase; this typically involves retraining the model using the data responsible for the performance decline. However, detecting drift based on error rates depends on having real-time access to the true data labels to compare with predictions. The availability of the ground truth data labels is also often needed to apply supervised algorithms during the training phase, which is triggered by the concept drift adaptation phase. The second macro-category is based on the feature distribution of the input data (data distribution-based drift detection). These algorithms require the user to define the size of both the historical time window and the new data window. However, it is generally reported that these approaches have a higher computational cost than the error-based ones. Such techniques are often created by adapting the previously mentioned mono-dimensional algorithms designed for error-based concept drift detection. For example, some works in the literature propose an adaptation of KSWIN to perform multidimensional detection.

The approach proposed in Togbe et al. (2021) suggests applying the KSWIN method to each feature individually. However, this strategy has a significant drawback: small simultaneous variations across multiple features can lead to concept drift, which may remain undetected if the system only examines features one at a time. To address this limitation, the authors of Agate et al. (2022) propose creating a synthetic feature that captures changes across all features collectively. The KSWIN method is then used to analyze distribution shifts in all features, including the synthetic one. If a drift is detected in any feature, the system triggers the Concept Drift Adaptation phase.

Although interest in developing concept drift detection algorithms has grown significantly in recent years, their unification into a common mathematical framework has received comparatively little attention. The authors of Hinder et al. (2024), recognizing this gap, propose a formal characterization of concept drift detectors and a set of properties they should satisfy. The underlying idea is to treat drift detectors as statistical hypothesis testing tools that distinguish the null hypothesis, that $D_t = D_s$ for all time points t and s , from the alternative that there exist t and s with $D_t \neq D_s$. Formally, a drift detector takes as input a data sample S of size n drawn from the data stream and returns a Boolean value indicating the presence or absence of drift. It is defined as a sequence of measurable functions:

$$A_n : (T \times \mathcal{X})^n \rightarrow \{0, 1\}. \quad (3)$$

Building on this formalization, the authors of Hinder et al. (2024) describe two key properties that concept drift detectors should satisfy:

- *surely drift-detecting*: the detector converges to the correct decision asymptotically. Formally, A is surely drift-detecting if, for every distribution process D_t and every $\delta > 0$, there exists N such that for all $n > N$:

$$\mathbb{P}_{S \sim D^n} [A_n(S) = \mathbf{1}[D_t \text{ has drift}]] > 1 - \delta. \quad (4)$$

- *validity*: the ability to control the false positive rate independently of the underlying data stream. Formally:

$$\begin{aligned} \limsup_{n \rightarrow \infty} \sup_{\substack{D_t \in \mathcal{D} \\ D_t \text{ has no drift}}} \mathbb{P}_{S \sim D^n} [A_n(S) = 1] \\ < \inf_{\substack{D_t \in \mathcal{D} \\ D_t \text{ has drift}}} \liminf_{n \rightarrow \infty} \mathbb{P}_{S \sim D^n} [A_n(S) = 1]. \end{aligned} \quad (5)$$

In other words, once a sufficient number of samples is provided, the probability of a false alarm falls below a controllable threshold regardless of the underlying data distribution. A is universally valid if this property holds for all possible distribution processes, i.e., when $\mathcal{D}$ is the set of all distribution processes.

Two types of error correspond directly to these two properties. The first is a false alarm: no drift is present, yet the detector raises an alert. The second is a missed detection: drift is present, but the detector fails to identify it. The latter is inherently tied to the *surely drift-detecting* property, since no fixed sample size can guarantee detection when the drift effect is subtle; if the effect is too small to impair system performance, a missed detection may not be practically consequential. Since the *surely drift-detecting* property cannot be guaranteed in general, the practical focus shifts to satisfying the *validity* property by controlling the false alarm rate. In practice, however, a trade-off inevitably arises: the more sensitive the detector is to distribution changes, the more likely it is to correctly identify genuine drift, but at the cost of an increased false alarm rate.

Error rate-based concept drift detectors are widely adopted in part because the failure to satisfy the *surely drift-detecting* property is tolerable in this context: if drift has not induced a measurable shift in the model's error distribution, it has not significantly impaired system performance and can therefore go undetected without consequence. Conversely, although the hypothesis test allows the *validity* property to be satisfied by adjusting detector sensitivity, it monitors the model's error distribution rather than the original data distribution. As a consequence, false alarm rates tend to increase substantially due to continuous fluctuations in model performance arising from data noise and class imbalance, even in the absence of genuine concept drift.

Despite the considerable number of false alarms they generate, error rate-based detectors remain widely adopted because they are typically paired with adaptive strategies that update the model continuously, even in the absence of drift, thereby departing from the framework illustrated in Fig. 3. Since ground truth labels are required for detection, these labels can subsequently be reused for adaptation. The presence or absence of detected drift then serves to select between different update policies: a more conservative strategy when no drift is detected, and a more aggressive adaptation towards the new data distribution when drift is confirmed. Consequently, false alarms are treated as a secondary concern. The principal limitation of these approaches, as discussed above, remains the assumption that ground truth labels are available in real time for all incoming data points.

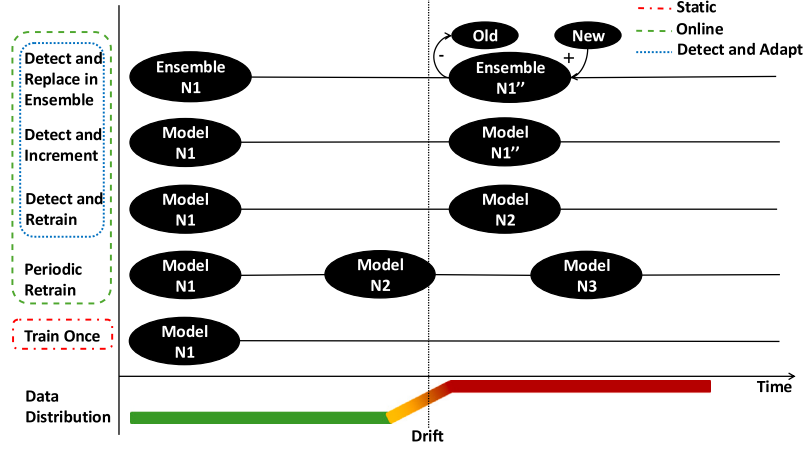


Fig. 4. Concept Drift Adaptation strategies.

During the *Concept Drift Adaptation* phase, possible strategies can be distinguished along two dimensions: the type of adaptation, which is closely tied to the underlying machine learning model, and the triggering mechanism. A strategy may be activated periodically after each data batch regardless of whether drift has been detected (*Periodic* strategy), or exclusively upon a positive detection signal from the concept drift detector (*Detect and Adapt* strategy). Three main adaptation types can be identified:

1. *Retrain*: trains a new model from scratch and discards the old one;
2. *Replace in Ensemble*: trains a new model and incorporates it into an existing ensemble, requiring policies for both weighted voting and model replacement (Zhou et al., 2020);
3. *Increment*: adapts the current model continuously through incremental learning, requiring algorithms capable of online training such as random forests and gradient boosting (Bayram et al., 2020).

A fourth hybrid strategy, proposed in Agate et al. (2024), explicitly addresses recurring concept drift by maintaining a set of models in memory. At each prediction step, the model best suited to the current concept is selected, enabling the reuse of previously trained models when a known concept recurs. A retraining phase is triggered only when no model in memory is adequate for the new concept.

As illustrated in Fig. 4, although *Detect and Adapt* strategies may appear to be a subcategory of their periodic counterparts, the difference is substantial. Periodic adaptation incurs continuous computational costs regardless of whether drift has occurred; moreover, retraining on noisy or imbalanced batches in the absence of genuine drift can degrade model performance. *Detect and Adapt* strategies avoid these costs, though their performance may be affected by missed or delayed detections.

The proposed HOIDS system employs a *Detect and Retrain* strategy; however, unlike state-of-the-art systems, its hybrid multi-layer architecture enables a flexible trade-off between labeling cost and computational overhead while maintaining performance comparable to the top-performing systems across the adaptation categories listed above. A distinctive feature is the adoption of the proposed MPCAWIN, an unsupervised multi-dimensional concept drift detection method based on the statistical distribution of input features, which requires no ground-truth labels during the detection phase and allows precise tuning of its sensitivity in accordance with the validity property.

4. HOIDS architecture

This section describes the multi-layer architecture of the hybrid online IDS we propose, named HOIDS, and depicted in Fig. 5.

HOIDS continuously monitors online network data streams and, based on its current models, detects both known and zero-day attacks. Concurrently, it stores and analyzes data to detect concept drifts and possibly retrain its models.

It is worth noting that working with data streams introduces additional challenges, such as managing time and memory constraints and addressing concept drift.

To address memory constraints, data streams are managed in batches, and only a small number of batches are saved using a fixed-size sliding window (W). To address temporal constraints, data processing is performed sequentially, minimizing the latency between data acquisition and analysis. This approach, also known as *test-then-train*, is commonly referred to as *Prequential Evaluation*.

When a new record becomes available, the system immediately evaluates it using the current machine learning-based supervised *Classifier*, as shown by the green dotted path in Fig. 5.

After the evaluation phase, the records are collected into batches of fixed size bs , and t of these batches are then stored in a sliding window W for potential concept drift detection and adaptation (orange continuous path in Fig. 5).

Specifically, HOIDS uses the *Detect and Retrain* strategy, as discussed in Section 3. When the concept drift is detected by the *Concept Drift Detection* module, the adaptation phase is triggered, leading to the training of a *New Classifier* with the records present in W . The *Active Learning (AL)* module assists the drift adaptation phase by minimizing the number of records that need to be manually labeled by human experts. The *AL* module infers the statistical distribution of benign traffic in the data used for retraining. It then requests labeling only for records that significantly deviate from this distribution (uncertain data), greatly reducing the expert's workload. The uncertain data includes malicious data that naturally deviate from benign traffic and benign data that are difficult to distinguish from malicious. All other records are considered benign for the retraining phase. Ideally, the *AL* module should correctly label as much benign data as possible without mistakenly including malicious records. Further details on the *AL* module are provided in Section 4.2.

The workflow of HOIDS is depicted in Fig. 5 and outlined step by step in Algorithm 1.

First, some variables are initialized in lines 1–4: $\hat{Y}$ is the list containing the outputs of the system for each evaluated record, RD is the list of the labeled data for retraining, B is a data batch. The boolean variable c_d indicates whether drift has been detected. In lines 5–7, according to the prequential evaluation strategy, new input data is evaluated immediately with the *Classifier*, and the results are appended to $\hat{Y}$. Each incoming record x is then appended to B (line 8). If B is filled up (line 9), a new batch is added to W using the sliding window mechanism (lines 10–13). When adding a new batch, *Concept Drift*

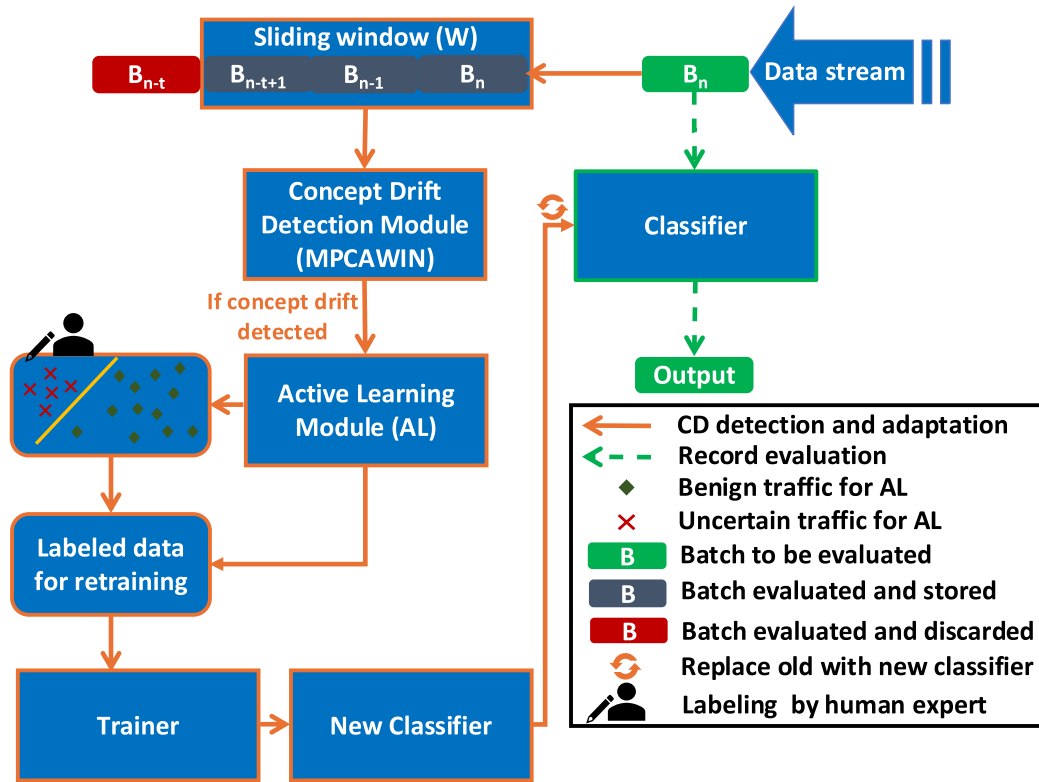


Fig. 5. Architecture of HOIDS.

Detection module verifies the eventual occurrence of concept drift (line 14). If drift is detected (line 15), the Concept Drift Adaptation phase is activated (lines 16–23). The AL module classifies and splits the W records into benign data (D_{benign}) and uncertain data ($D_{uncertain}$) in line 16, and the D_{benign} are added to the RD (line 17). A human expert is required to label uncertain data (line 18), so they can be added to the labeled dataset for retraining. This dataset is then exploited to train a *New Classifier* (line 21).

4.1. Concept Drift Detection module: MPCAWIN

As discussed in Section 2, most of the approaches available in the literature, including (Celik and Vanschoren, 2021; Montiel et al., 2020), use mono-dimensional concept drift detection algorithms (CDD_{mono}) based on error rates.

These methods represent each prediction as either 0, when it matches the ground-truth, or 1, when it differs, managing these values through a sliding window. This approach has three main limitations. First, it requires timely access to the full set of ground-truth data for comparison with system classifications, which is unrealistic in the IDS context. Second, it has low resilience to data noise; even a small number of misclassifications caused by noisy data can increase false positives and lead to unnecessary adaptation phases, as will be highlighted in Section 5.

Third, the error-rate approach does not handle virtual drift (Fig. 1-a), since a change in the feature space distribution that does not lead to an immediate degradation in system performance goes undetected and unaddressed: error-rate-based detectors monitor changes in $P(y|X)$ (Eq. (1)) and are therefore oriented towards actual drift (Fig. 1-b). Conversely, distribution-based detectors monitor changes in $P(X)$ and therefore primarily address virtual drift, but in turn fail to handle actual drift: a change in the labels to be assigned to incoming samples that is not accompanied by a shift in the feature distribution cannot be detected.

The importance of handling one drift source over another is closely tied to the application domain. In contexts such as spam or malware detection, handling actual drift is critical. Consider an application initially catalogued as benign that is later found to conceal malware, or an email first flagged as spam by the automated model and subsequently restored by the user. From a feature space perspective, the samples undergo no distribution change, yet the model must incorporate the updated label information to ensure correct classifications going forward. In the network IDS context in which this work is situated, actual drift is largely absent or irrelevant: it would correspond to network flow records whose ground-truth labels change without any accompanying shift in the feature distribution, an unlikely scenario in practice. Virtual drift, on the other hand, is the dominant and most critical phenomenon: benign and malicious traffic patterns evolve continuously, and even when an initial distribution shift does not immediately degrade model performance, it may bring samples progressively closer to the decision boundary, eventually triggering a sudden drift (Fig. 1-b). Detecting and adapting to virtual drift early enables the system to cope with gradual and incremental drifts (Fig. 2) and prevent abrupt performance degradation, which represents the primary threat in online IDS deployment.

To overcome these limitations, the data distribution-based Concept Drift Detection module we propose here detects drift by analyzing changes in the statistical distribution of input features. This method, named *Multidimensional PCA-based Windowing (MPCAWIN)*, consists of three main components: the *Multidimensional PCA Comparator (MPC)*, the *Statistical Anomaly Detector (SAD)*, and the *Monodimensional Concept Drift Detector (CDD_{mono})*. Their roles and interactions are illustrated in Fig. 6.

MPC is initialized with a batch of data from which it extracts the first feature representation. This becomes the *Current Concept (B_i) Feature*, which will be updated over time as new concept drifts are detected (step 1 in Fig. 6).

For each new batch of data (B_n), MPC extracts a corresponding feature representation, referred to as the B_n Feature, which is then

Algorithm 1: HOIDS workflow

Input :

Δ : data-stream;
 bs: batch size;
 W: sliding window;
 t: number of batches stored in W;
 CDD: concept drift detection module;
 AL: active learning module;
 Classifier: machine learning-based supervised classifier;
 HI: human labeling phase;
 Trainer: machine learning training phase.

```

1  $\hat{Y} \leftarrow []$ ;  $\triangleright$  the list of system's outputs for each evaluated record
2  $RD \leftarrow []$ ;  $\triangleright$  the list of the labeled data for retraining
3  $B \leftarrow []$ ;  $\triangleright$  the list of records in a batch
4  $c_d \leftarrow False$ ;  $\triangleright$  boolean that indicates the concept drift detection
5 for  $x \in \Delta$  do
6    $\hat{y} \leftarrow \text{Classifier.evaluate}(x)$ 
7    $\hat{Y}.append(\hat{y})$ 
8    $B.append(x)$ 
9   if  $len(B) == bs$  then
10     if  $len(W) == t$  then
11        $W.remove\_oldest\_batch()$ 
12     end
13      $W.append(B)$ 
14      $c_d \leftarrow CDD.detect\_drift(B)$ 
15     if  $c_d == True$  then
16        $(D_{benign}, D_{uncertain}) \leftarrow AL.evaluate(W)$ 
17        $RD.append(D_{benign})$ 
18        $D_{labeled} \leftarrow HI(D_{uncertain})$ 
19        $RD.append(D_{labeled})$ 
20        $New\ Classifier \leftarrow Trainer.train\_classifier(RD)$ 
21        $Classifier \leftarrow New\ Classifier$ 
22        $RD \leftarrow []$ 
23     end
24      $B \leftarrow []$ 
25   end
26 end

```

compared to the *Current Concept* (B_i) *Feature*, through the evaluation of a dissimilarity score $\gamma_{i,n}$ (step 2 in Fig. 6). These scores are then processed by the SAD module, which maintains a sliding window A containing the m most recent dissimilarity values. If the dissimilarity score $\gamma_{i,n}$ is greater than a given threshold α (step 3 in Fig. 6) it is treated as noisy data, and a value of 1 is appended to the sliding window successively processed by the CDD_{mono} component (step 4a in Fig. 6). Otherwise, the score is considered consistent with the distribution, appended to the sliding window A, and a 0 is appended to the sliding window of CDD_{mono} (step 4b in Fig. 6). After the update, the threshold α is recalculated as a function of the mean and standard deviation of the values stored in the window A, i.e., $\mu(A) + c\sigma(A)$ (step 5 in Fig. 6). As a result, SAD generates a stream of binary labels (1 or 0), indicating whether each batch significantly deviates from the current concept.

After each new value is appended to the sliding window, CDD_{mono} (through, the KSWIN method Raab et al., 2020) analyzes the resulting distribution and determines whether a concept drift has occurred (step 6 in Fig. 6).

KSWIN operates on a fixed-size sliding window of size sw (Fig. 7). Since the Kolmogorov–Smirnov test requires two data distributions as input, two sub-windows are extracted: the *New Concept* window, containing the most recent ss samples (from index $sw - ss + 1$ to sw), and the *Last Concept* window, of the same size ss , uniformly sampled from the remaining $sw - ss$ historical samples. The uniform sampling distribution is:

$$U(x \mid 1, sw - ss) = \frac{1}{sw - ss}. \quad (6)$$

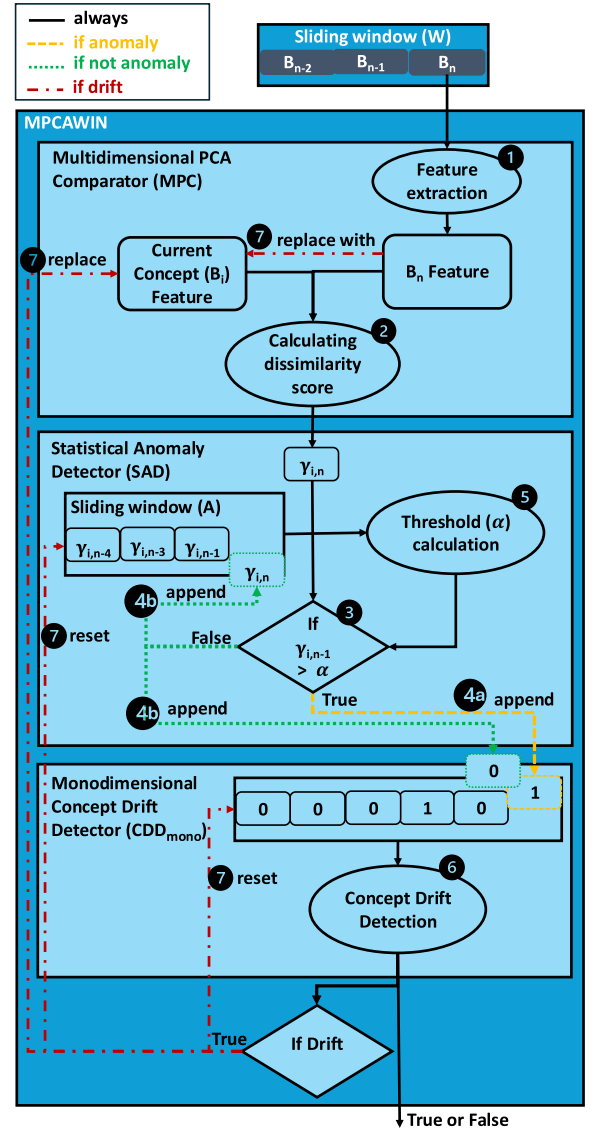


Fig. 6. MPCAWIN architecture.

Denoting by L the Last Concept window, R the New Concept window, and SW the full sliding window, we have:

$$L = \{x_i \in SW \mid i < sw - ss + 1 \wedge p(x) = U(x_i \mid 1, sw - ss)\}. \quad (7)$$

Concept drift is detected when the Kolmogorov–Smirnov statistic exceeds the detection threshold:

$$dist(R, L) > \sqrt{-\frac{\ln(\lambda)}{ss}}. \quad (8)$$

that is, when the dissimilarity between R and L is large enough to reject the hypothesis that both samples are drawn from the same distribution. The λ parameter controls the significance level of the Kolmogorov–Smirnov statistical test and thereby the sensitivity of the detector in accordance with the *validity* property.

When concept drift is detected, all parameters of MPCAWIN are reset, and the *Current Concept* (B_i) *Feature* is replaced with the B_n *Feature* (step 7 in Fig. 6). This reset ensures the system remains synchronized with the evolving data distribution, maintaining the reliability of subsequent comparisons and minimizing the risk of false detections.

The $\alpha = \mu(A) + c\sigma(A)$ formulation is motivated by Chebyshev's inequality (Marshall and Olkin, 1960), which states that for any distribution with finite mean and variance, the probability that a random

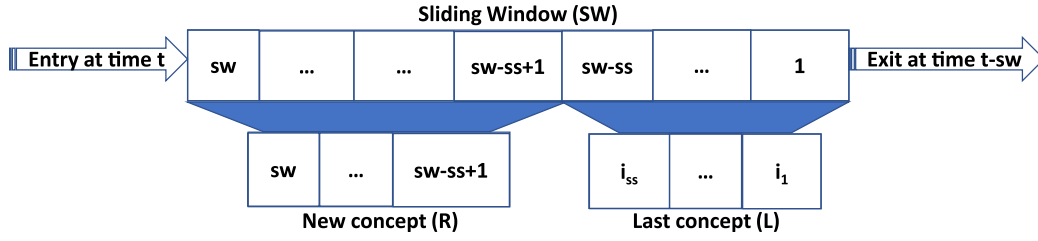


Fig. 7. KSWIN Data Retrieval phase with sliding window.

variable deviates from its mean by more than c standard deviations is bounded above by $1/c^2$, regardless of the underlying distribution. For the default value $c = 2$, this yields a conservative upper bound of 25% on the probability that a non-drifting batch is incorrectly flagged as anomalous by the SAD component. Crucially, this bound holds without any assumption on the shape of the dissimilarity score distribution, which is important given that the distribution is not known a priori and may vary across datasets and deployment scenarios, consistently with the *universal validity* property (Section 3), which requires that the false positive rate be controllable regardless of the underlying data stream.

Sporadic anomalous flags produced by SAD do not directly trigger drift detection: they are passed to a valid CDD_{mono} such as KSWIN, which requires a sustained change in the distribution of the binary stream before declaring drift. This two-stage design provides an additional layer of robustness against false positives, making the choice of c less critical than it would be in a single-threshold system.

As discussed in Section 3, MPCAWIN thus enables control of the trade-off between sensitivity, false positive rate, and detection delay, in accordance with the *validity* property.

The sensitivity analysis reported in Table 3 further confirms that values of c in the range $[2, 6]$ all yield correct detection with a delay of 5 batches, supporting the robustness of this design choice. From a methodological standpoint, the MPCAWIN components can be mapped onto the abstract phases of concept drift detection introduced in Section 3. Compared to existing mono-dimensional concept-drift detectors, MPCAWIN adds three meta-phases: *Data Retrieval*, *Data Modeling*, and *Statistic Test Calculation*, in order to derive a mono-dimensional distribution for CDD_{mono} . At a high level, *MPC* implements the *Data Retrieval* and *Data Modeling* phases; *SAD* corresponds to the *Statistic Test Calculation* phase, and CDD_{mono} is used as the *Hypothesis Test*.

The *Data Modeling* phase of the *MPC* component is inspired by previous works (Shao et al., 2014; Wahab, 2022), where a Standard Scaler and PCA (Wold et al., 1987) are applied to data batches at times t and $t-1$ to extract eigenvalue vectors. The angle between these vectors is used as a dissimilarity score, which is then compared against a fixed threshold; concept drift is detected if the angle exceeds this threshold.

The original approach has three main limitations. First, it fails to detect incremental drift over time, since small but frequent changes between consecutive batches will not cause drift detection. Second, the method relies on a fixed threshold to determine whether changes in variance between two batches are due to concept drift. However, over time this threshold may become unsuitable, especially as the data distribution evolves. The third limitation is the method's low robustness to noise, as it detects concept drift whenever there is a sudden change in variance between two batches, which could be caused by the mere presence of noisy data.

MPCAWIN addresses these limitations through a set of specific strategies, detailed below. First, the *Data Retrieval* phase of the *MPC* component retains in memory the last batch B_i that was set as the current concept. All subsequent comparisons are made against the B_i *Feature*, enabling the detection of incremental concept drift. Second, the fixed threshold is replaced by a dynamic one, computed during the *Statistic Test Calculation* phase implemented by the *SAD* component. Finally, MPCAWIN increases robustness to noise through the *Hypothesis*

Test phase performed by CDD_{mono} , which operates on the distribution generated by the previous level. Since data noise is typically sporadic and short-lived, it may occasionally cause an anomalous dissimilarity score. However, such isolated deviations are unlikely to generate a sustained stream of values 1, and thus rarely induce a significant change in the distribution monitored by CDD_{mono} .

The *MPC* component of MPCAWIN applies a Standard Scaler to each data batch before computing the PCA decomposition, ensuring that all features are normalized to zero mean and unit variance prior to the eigenvalue extraction. This preprocessing step mitigates the sensitivity of PCA to differences in feature magnitude, which is particularly important in network traffic datasets where features can span several orders of magnitude. In addition, although standard PCA is inherently limited to capturing linear structure, in the context of MPCAWIN the role of PCA is not to reconstruct the full data distribution but to extract a compact and comparable feature representation of each batch. In this setting, linear projections have proven sufficient to capture the distribution shifts relevant to concept drift detection in the evaluated scenarios, as will be shown later in Section 5.5.

4.2. Active learning module

The role of the *active learning module* (AL) is as a pre-filter to identify those records that may require manual review before the retraining phase, with the aim of reducing the amount of data that needs to be inspected by human experts. Due to the online behavior of IDSs, it is unrealistic to assume that ground-truth labels for the current window (W) are available in time for training. Therefore, this step should be performed in an unsupervised manner, before expert intervention.

An AutoEncoder (AE) has been selected as the base model for the AL module, not to directly identify anomalies, as in traditional anomaly detection pipelines, but to flag uncertain records that could benefit from human review.

The typical use of AE in the literature focuses on offline anomaly detection, and it is based on strong assumptions regarding the availability of ground-truth labels, which are necessary to construct a training set composed only of benign samples, and to perform fine tuning of a decision threshold to distinguish benign and malicious samples. In such settings, the AE is used to compare each reconstruction error $re(i)$ with the chosen threshold τ .

The reconstruction error is defined as follows:

$$re(i) = \|x_i - d_\theta(e_\phi(x_i))\|, \quad (9)$$

where x_i is the original data point, d_θ is the decoding function, and e_ϕ is the encoding function. The assumptions behind this approach are not valid in online scenarios where ground-truth labels are unavailable, and where data may include both benign and malicious instances. In this work, τ is computed exclusively based on the statistical characteristics of the reconstruction error distribution and recomputed at each adaptation phase, thus defining a more realistic and robust strategy.

The architecture of the AutoEncoder is as follows. The encoder consists of three hidden layers with dimensions 512, 256, and 64, while the decoder includes two hidden layers of size 256 and 512. All layers, except the output, use the Gaussian Error Linear Unit (GELU) activation

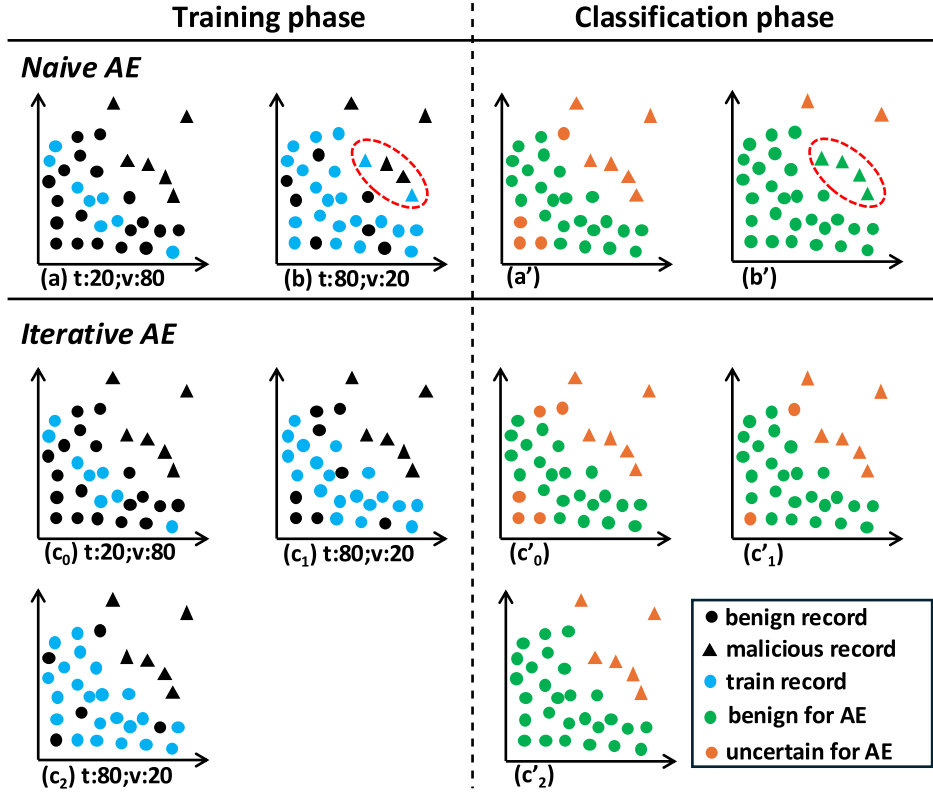


Fig. 8. Two-dimensional examples of Naive and Iterative AutoEncoder for the AL module.

function, while the output layer uses a linear one. The Mean Absolute Error (MAE) of the reconstruction error is used as the loss function during training:

$$MAE = \frac{\sum_{i=0}^N re(i)}{N}. \quad (10)$$

Since τ is applied to the reconstruction error to filter uncertain data, it is important to consider how anomalies are distributed within a batch of observations. In the context of intrusion detection, it is reasonable to assume that, within a given window, the proportion of malicious data is relatively low compared to the total number of observations. However, even when malicious records are rare, their presence can still have a significant impact on the threshold calculation.

The impact of malicious records strongly depends on how the data is split between training and validation sets. If a non-negligible number of such records are included in the training set, they can compromise both the learning process of the AE and the accuracy of the threshold τ .

Fig. 8 shows two examples illustrating the behavior of a Naive AutoEncoder (*Naive AE*) under different training-validation splits.

In this work, the threshold τ is set to the median of the reconstruction error distribution computed over the training set, rather than derived from its mean and standard deviation as is typically done in the literature ($\tau = \mu(re) + \eta\sigma(re)$). The key motivation is robustness to outliers: in an online IDS setting, the training set used by the AL module at each adaptation phase contains an unknown mixture of benign and malicious records, with malicious samples typically producing significantly higher reconstruction errors. Even a small number of such samples can substantially inflate the mean and shift percentile-based estimates, such as the 95th percentile, making them unreliable as thresholds. The median, by contrast, is insensitive to a minority of high-error outliers, provided that the majority of the training set consists of benign records. This is a reasonable assumption in the IDS context.

Furthermore, the choice of the median reflects the conservative stance required by the AL module, whose goal is to correctly identify

as much benign traffic as possible while minimizing false negatives. By construction, the median ensures that approximately half of the training records are labeled as benign by the autoencoder, leaving the remainder for expert annotation. As τ increases, more records are labeled as benign, reducing the burden on the expert but increasing the risk of misclassifying malicious traffic as benign; conversely, a lower τ flags more records as uncertain, reducing false negatives at the cost of greater expert involvement.

This trade-off has been validated through experimental evaluations, as discussed in Section 5.

The two-dimensional *Naive AE* examples in Fig. 8 shows the effect of different training-validation splits: one with 20% training and 80% validation (Fig. 8-a,a'), and the other with the opposite configuration (Fig. 8-b,b'). These configurations show the trade-off between the number of uncertain records and the number of malicious samples misclassified as benign.

In Fig. 8-a, using only 20% of the dataset for training likely results in a set composed entirely of benign records (or, at least, a negligible number of malicious ones), thus minimizing the risk of false negatives. However, during prediction, many benign records that differ significantly from the training subset are likely to be considered uncertain (Fig. 8-a'). In Fig. 8-b, using 80% of the records for training mitigates this issue. However, it also increases the likelihood of including malicious samples in the training set, which may lead to more false negatives, even when using a conservative threshold (Fig. 8-b'). In this case, the presence of a larger number of malicious samples during training affects the AutoEncoder's ability to distinguish between benign and malicious records, reducing the gap between their reconstruction errors and making the threshold τ ineffective for accurate classification.

In light of the limitations shown by the Naive AE model, an Iterative approach (*Iterative AE*) is proposed, in this work, with the goal of minimizing false negatives and reducing the number of records requiring expert review. *Iterative AE* adapts both the data splitting strategy and the threshold τ computation over successive iterations. Each training

phase in the *Iterative AE* acts as a semi-supervised step, including those benign records identified from previous iterations. This process is fully automatic and does not require external intervention or ground-truth labels.

The first iteration, illustrated in Fig. 8-c₀,c'₀, follows the same approach as the *Naïve AE* configuration shown in Fig. 8-a,a'. Subsequent iterations, shown in Fig. 8-c₁, select 80% of the newly identified benign records for training, enhancing the training set with additional benign data while avoiding the inclusion of records considered malicious.

After this training phase, the threshold τ is calculated as $\tau = \mu(re) + 3\sigma(re)$, since using the median would not lead to the identification of additional benign records compared to the previous iteration. This updated threshold allows the identification of benign records that differ slightly from those previously recognized, reducing false negatives (Fig. 8-c'₁).

The expected behavior is a gradual decrease in the number of newly classified benign records across iterations. This approach reduces the number of benign records that remain uncertain at the end of the process, while also lowering the number of false negatives (Fig. 8-c₂,c'₂).

The iterative process terminates under two conditions. The primary condition is natural convergence: the process halts when no new benign records are identified in the current iteration, meaning that the refined threshold τ and the enriched training set no longer produce additional classifications compared to the previous iteration. The secondary condition is a safeguard against false negative propagation: if the number of newly identified benign records does not follow the expected decreasing trend across iterations, the process is halted early and the output of the previous iteration is used as the final result. This prevents a situation where malicious records begin to be incorrectly absorbed into the benign set, which would compromise both the threshold estimation and the quality of the retraining data. Together, these two conditions ensure that IAE always terminates in a finite number of iterations and does so without requiring any external stopping criterion or ground-truth supervision.

The proposed AL module differs fundamentally from standard active learning sampling strategies such as uncertainty sampling: it is a semi-supervised algorithm in which the AutoEncoder plays a dual role simultaneously. On one hand, it acts as an automatic classifier, correctly labeling the majority of benign records in the retraining window without any expert intervention. On the other hand, it identifies the uncertain records to be submitted for manual review. Furthermore, in the IAE approach these two roles are tightly coupled through the iterative process: each iteration enriches the training set with newly identified benign samples, refining the reconstruction error distribution and progressively reducing the number of uncertain records. The process halts automatically when no new benign records are identified, or when the expected decreasing trend in newly classified samples is violated, preventing false negatives from propagating into the training set. This end-to-end mechanism, which requires no ground-truth labels and no external stopping criterion, is what distinguishes IAE from standard active learning approaches, which treat the classifier and the querying strategy as separate components and the query budget as an input parameter.

4.3. Classifier

The *Classifier* uses the XGBoost (Extreme Gradient Boosting) model (Chen and Guestrin, 2016) in supervised classification mode. This choice is due to its robust performance described in the IDS literature. The assessment of the effectiveness of XGBoost, as well as its variants used in other studies, is detailed in Section 5.

XGBoost is an ensemble technique based on Boosting Tree models that improves the predictive performance of classical models by training multiple classifiers and combining their predictions $\hat{y}_i$ for a given input x_i (Sagi and Rokach, 2018).

In particular, during the prediction phase, the final prediction $\hat{y}_i$ for a sample x_i corresponds to the sum of the predictions for each tree f_k in the ensemble F :

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in F. \quad (11)$$

For each Concept Drift Adaptation (CDA) stage, the training dataset includes the batches contained in W that caused the drift. In this work the data is used in its original form, without preprocessing, and is labeled based on the AL module and human expert intervention.

Before the training phase, the technique of oversampling the malicious class is used, as suggested in Yang et al. (2022), increasing the number of records of that class to a thousand records. This approach mitigates the impact of class imbalance on the performance of the supervised model.

This way, the old *Classifier* is eliminated and a new one is trained and it is used to evaluate all future records until the next CDA stage.

5. Experimental evaluation

This section describes the performance evaluation of HOIDS and its comparison with both static and online state-of-the-art works. The evaluation, performed on real-world datasets, aims to assess the need to address the concept drift problem, the strengths and weaknesses of the compared systems, and the validity of the HOIDS approach.

To this end, the comparison has been performed by adopting a comprehensive suite of metrics namely Accuracy (Acc.), Precision (Prec.), Recall (Rec.) F1-score, false positive rate (FPR), and false negative rate (FNR).

5.1. Dataset description

The evaluation has been performed on the improved versions (Liu et al., 2022) of two real network traffic datasets, CIC-IDS-2017 and CSE-CIC-IDS-2018 (Sharafaldin et al., 2018), which are considered primary benchmarks for NIDS. Starting from these public datasets, three subsets have been created to support specific experiments:

- *Impr2017* and *Impr2018*: created by undersampling and shuffling the records of the original CSV files, including all types of benign traffic and attacks present in the Improved-CIC-IDS2017 and Improved-CIC-IDS2018 datasets, respectively. The original multi-class labels have been converted in binary labels (normal/attack), with attacks constituting about 3% of the total records. These two datasets contain the intersection of features of the original datasets, and are used to test the intra-dataset performance of static systems (Section 5.6).
- *ImprTotal*: created by concatenating records of *Impr2017* and *Impr2018*, without further shuffling, to create a dataset that exhibits a sudden drift in the transition between the two. This dataset is used to test the inter-dataset performance of static systems and online performance of online systems, as described in Section 5.6.

The drift used in this work is based on the concatenation of two real network flow-based traffic datasets, CIC-IDS-2017 and CSE-CIC-IDS-2018, collected in different time periods and network environments, with partially different attack types and traffic characteristics, but with the same packet analyzer and statistical feature extractor. This property is important to ensure that the drift in the distribution of features is realistic and not artificially induced by a different method of pre-processing network communication packets.

It is important to note that, even though the two original datasets are considered benchmarks for NIDS-based research, they present some weaknesses for online approaches. They often contain a disproportionately high percentage of anomalous records compared to what an IDS

Table 1
HOIDS hyperparameters.

Parameters	Info	Component (Python library)	Module
<i>bs</i> : 2000 <i>PCA</i> : default <i>SC</i> : default	Size of compared batches PCA default hyperparameters Standard Scaler default hyperparameters	MPC (scikit-learn)	MPCAWIN
<i>m</i> : 10 <i>c</i> : 2	Size of sliding window A $\alpha = \mu(A) + c\sigma(A)$	SAD (numpy)	
<i>KSWIN</i> : recommended	Type of CDD	CDD _{mono} (scikit-multiflow)	
<i>optimizer</i> : ADAM <i>lr</i> : 0.001 <i>se</i> : 10 <i>patience</i> : 5 <i>ep</i> : 1000	Optimizer Learning rate Early Stopping start epoch Early Stopping patience parameter Maximum number of epochs	AutoEncoder (pytorch)	AL
<i>XGBoost</i> : default	Default hyperparameters	XGBoost (xgboost)	Classifier

would encounter in a real deployment, with attacks sometimes repeated for hours consecutively. Without undersampling and shuffling within each dataset, batches would alternate between predominantly benign and predominantly malicious traffic, producing apparent distribution shifts that reflect an unrealistic class distribution rather than genuine concept drift. Furthermore, although the data was collected from real network traffic, it was gathered over a few days in a controlled environment and therefore cannot capture the distribution shifts that arise from long-term changes in network topology, infrastructure, services, or attack strategies. For these reasons, ImprTotal was composed as described above.

The resulting drift then reflects genuine distribution changes arising from differences in network topology, infrastructure, and attack composition between the two collection campaigns. The absence of further shuffling at the boundary between the two datasets ensures that the temporal structure of the transition is preserved, producing a sudden drift that is representative of real-world scenarios such as network migration or the emergence of new attack families.

It is worth noticing that this setup captures a single sudden drift event, and that real deployments may involve more gradual, recurring, or overlapping drifts. This is a known limitation of the currently available benchmark datasets for network intrusion detection. As discussed in the conclusions, the datasets commonly used in the literature to test concept drift management are either artificially generated or, when based on real traffic, lack the combination of high-dimensional features, realistic class imbalance, and genuine distribution shifts that make ImprTotal suitable benchmarks for this work.

In the online experiments, a prequential evaluation strategy was adopted to ensure a methodologically rigorous and reproducible evaluation: each system is first trained on the same set of samples corresponding to the offline training dataset; each subsequent sample is then first classified and, if a drift adaptation phase is triggered, used to update the model, while preserving the exact order in which the network traffic arrives. This evaluation protocol reflects a realistic deployment scenario and constitutes a methodologically valid alternative to significance testing approaches designed for i.i.d. contexts. In fact, given the online and time-ordered nature of the evaluation, standard resampling techniques such as k-fold cross-validation are not applicable: shuffling the records would violate the temporal integrity of the data stream and produce an unrealistic assessment of the system's ability to detect and adapt to concept drift, which is the primary objective of the experimental evaluation.

5.2. Experimental setup

The AutoEncoder at the basis of the Active Learning module, developed using PyTorch, is trained through backpropagation using the Adam optimizer with a learning rate of 0.001. An early stopping mechanism has been adopted to enhance training efficiency and prevent overfitting, which is achieved by measuring the Mean Absolute Error

on the validation set, starting from the 10th epoch, and stopping the training phase if no improvement is observed for 5 epochs (patience parameter). The training is further bounded by a maximum duration of 1000 epochs.

For the *Classifier*, the XGBoost models are developed using the xgboost library with default hyperparameters. The default XGBoost configuration represents a well-established and widely validated baseline across a broad range of classification tasks, and its adoption is consistent with the approach of the LCCDE (Yang et al., 2022) framework on which the classifier component of HOIDS is based.

Moreover, the use of default XGBoost parameters is a deliberate design choice motivated by the operational constraints of the online IDS context. In a standard offline setting, hyperparameter optimization is typically performed via cross-validation on a large labeled dataset. In an online drift-aware system, however, retraining is triggered dynamically upon drift detection and must operate on a limited set of labeled samples, making exhaustive hyperparameter search both computationally expensive and statistically unreliable. Using default parameters avoids this overhead entirely and ensures that the retraining phase remains practical under real deployment conditions. Preliminary experiments confirm that varying the key XGBoost hyperparameters produces no significant change in detection performance, indicating that the model is robust to the specific parameter choice in this domain. The oversampling technique is implemented using the SMOTE algorithm (Lemaître et al., 2017).

All experiments were performed with the same minimum training dataset size (w) to ensure fair comparisons under identical conditions. The value of w was experimentally set according to the static approach of LCCDE, as will be discussed later.

Considering that optimal parameter values of MPCAWIN cannot be defined in advance, a sensitivity analysis is conducted on batch size bs , size m of the sliding window A , and constant c for calculating the α threshold. As will be discussed in more detail in Section 5.5, it is possible to identify a range of recommended values for each parameter by considering a trade-off between robustness and detection timeliness. The parameters used for MPCAWIN have been set empirically in these ranges of values.

The number t of batches contained in the W window (Alg. 1) was set accordingly with bs . The hyperparameters for HOIDS are summarized in Table 1, grouped by module and reporting the Python library used for each component. Most of these parameters are either default values or commonly used in the IDS literature.

5.3. Static approach

The purpose of the experiments described in this section is to evaluate the performance of a static system in order to verify the representativeness of the training window size (w) that will be used in subsequent comparisons for online systems. This experimental evaluation is performed separately on the two datasets Impr2017 and

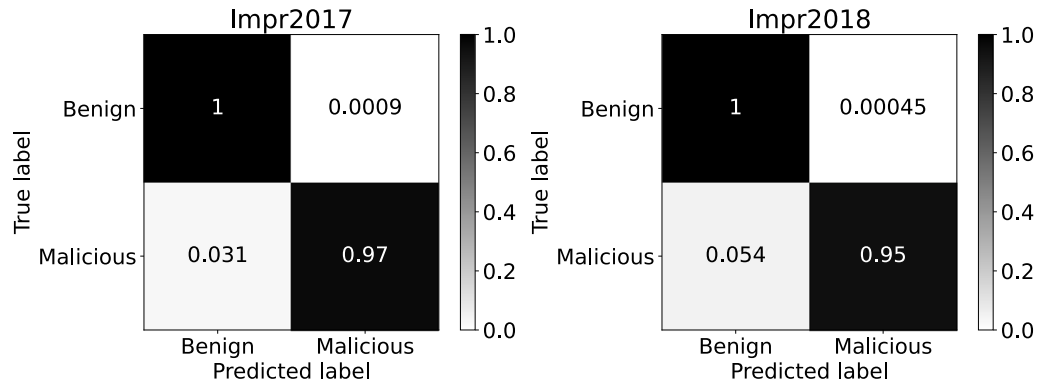


Fig. 9. Confusion matrices of LCCDE static system (training window size $w = 8000$) on Impr2017 e Impr2018 dataset.

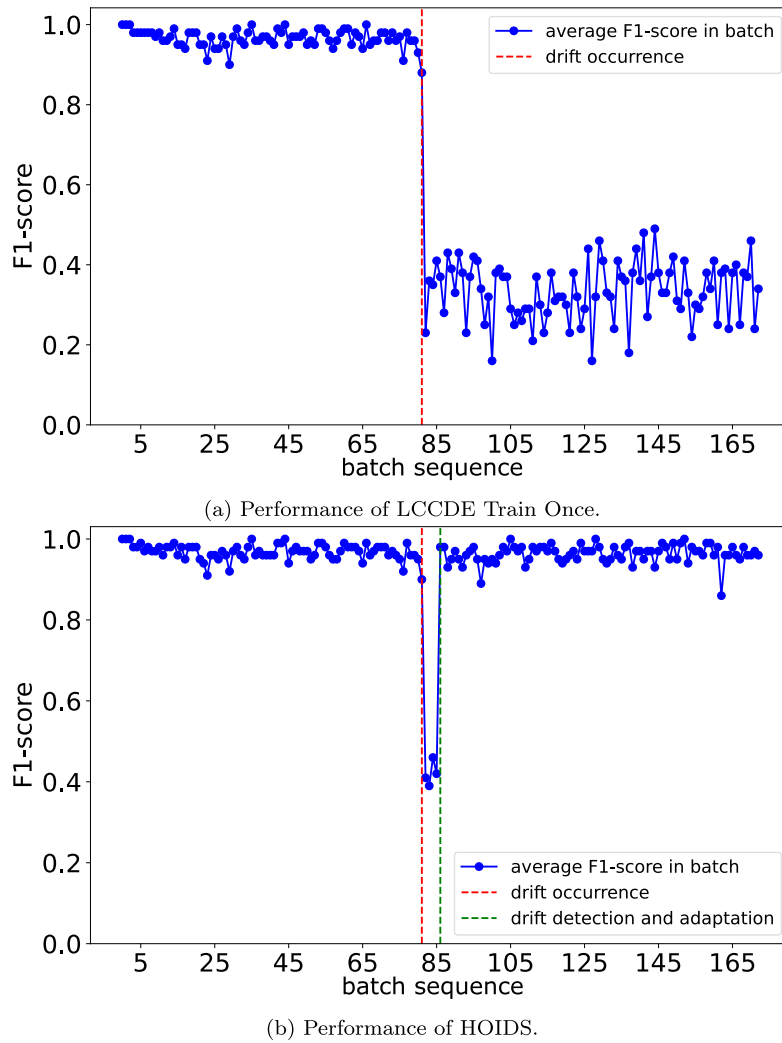


Fig. 10. Average F1-score in terms of batch sequence analyzed on ImprTotal dataset.

Impr2018 showing the intra-dataset performance that will serve as a comparison with the later inter-dataset online performance. To this end, the performance of the LCCDE (Yang et al., 2022) system is evaluated since it is considered representative of state-of-the-art static IDSs. The experiments were repeated by varying the size of the training window from 4000 to 20 000. The training window contains the first w records of the dataset without shuffling. The smallest size, achieving

comparable results to a classical 70-30 training-test split with shuffling, is 8000 records. As can be seen in Fig. 9, that shows the confusion matrices obtained by the static system LCCDE, respectively, on the Impr2017 and Impr2018 datasets with a training window size w of 8000 records, LCCDE yields remarkable results on both Impr2017 and Impr2018. A training window containing 8000 records is representative and appropriate for the ML-model, so this dimension w will be used

Table 2

Total results for compared systems in terms of Concept Drift Adaptation Strategy (CDA Strategy), Concept Drift Detection Strategy (CDD Strategy), percentages of records labeled by domain expert to obtain ground truth (GT%), Precision of concept drift detection ($Prec_{cdd}$), the concept drift detection delay in terms of number of batches related to the new concept evaluated before the correct detection of drift (Delay), Accuracy (Acc.), Precision (Prec.), Recall (Rec.), F1-score (F1), False Positive Rate (FPR), False Negative Rate (FNR).

System	CDA Strategy	CDD Strategy	GT%	$Prec_{cdd}$	Delay	Acc.	Prec.	Rec.	F1	FPR	FNR
LCCDE	Train Once	No	0%	–	–	0.987	0.945	0.564	0.706	0.001	0.436
LCCDE	Periodic Retrain	No	100%	–	–	0.998	0.976	0.951	0.963	0.001	0.051
AutoML	Detect and Retrain	Error Rate	100%	1/15	2	0.997	0.996	0.911	0.951	0.0001	0.097
AXGB	Per. Inc./Rep. in Ens.	Error Rate	100%	1/24	0	0.990	0.917	0.716	0.804	0.002	0.284
HOIDS _{BL}	Detect and Retrain	Data Distribution	2.35%	1/1	5	0.998	0.980	0.943	0.961	0.001	0.060
HOIDS _{NAE}			1.18%								
HOIDS			0.92%								

for the first training phase and in case of retraining required during the Concept Drift Adaptation phase of the online systems that will be compared in the following.

5.4. From static to online approach

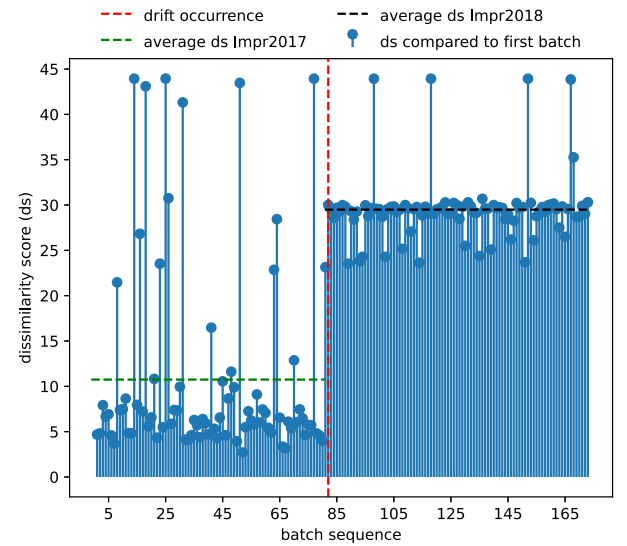
In order to verify that static approaches cannot adequately handle concept drift, this section describes the inter-dataset performance on ImprTotal of the static baseline, i.e. the LCCDE system used in an online approach with a Train Once CDA strategy and without a CDD strategy (Table 2). Fig. 10(a) shows the F1-score over time, in terms of batch sequence analyzed. In this case, LCCDE was tested on the ImprTotal dataset and trained with the first data window containing the first w Impr2017 network records. As shown earlier in Fig. 9, the system achieves excellent intra-dataset performance on Impr2017; accordingly, the F1-score remains high for all data batches containing Impr2017 records. Conversely, the system suffers a drastic performance degradation during the transition between Impr2017 and Impr2018, indicating the occurrence of concept drift (dashed red line).

The results of the analysis on the concept drift impact on LCCDE Train Once approach are summarized in Table 2. The recall (0.564) and FNR (0.436) scores highlight how concept drift has caused the misclassification of malicious records, which are incorrectly interpreted as benign. This issue is similar to scenarios in which networks experience zero-day attacks or in which attackers modify known attacks to evade the IDS (Fogla et al., 2006; Jordaney et al., 2017). Additionally, the precision (0.945) and FPR (0.001) scores indicate performance degradation in benign record classification compared to intra-dataset performance.

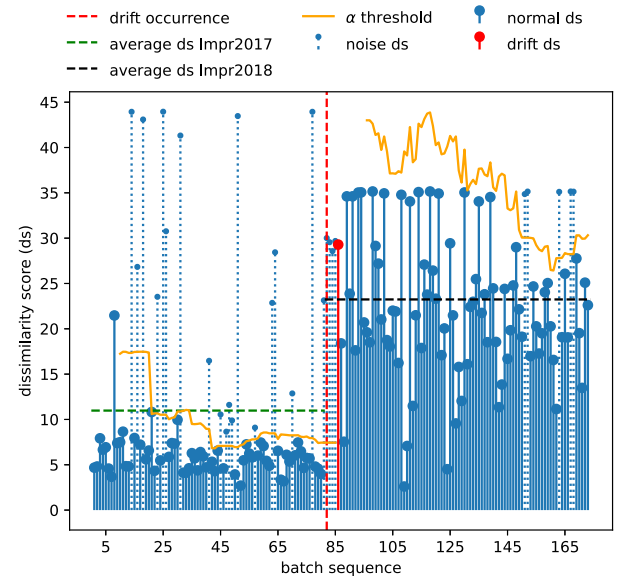
Some benign records are misclassified as malicious due to shifts in benign traffic patterns, such as changes in network topology, infrastructure, services, software updates, or employee behavior prompted by new corporate policies.

In order to evaluate the ability of HOIDS to react dynamically to the presence of the concept drift the online performance of HOIDS system are illustrated in Fig. 10(b). In contrast to LCCDE, the system only experiences temporary performance degradation due to concept drift, from its occurrence to its detection and subsequent adaptation phase (dashed green line). Retraining a new model with the data window that caused the drift, representing the new statistical distribution of benign and malicious data, allows the system to achieve optimal performance comparable to pre-drift conditions.

As shown in Table 2, all metrics of the HOIDS system outperform the LCCDE Train Once system. In particular, the F1-score of 0.961 compared to 0.706, the precision of 0.980 versus 0.945, the recall of 0.943 versus 0.564 and the FNR of 0.060 versus 0.436 (where lower is better). These results demonstrate the importance of accounting for potential concept drift in IDS operation, highlighting the need to move from a static to an online approach, detecting concept drift through a Concept Drift Detection (CDD) strategy and responding with an Adaptation strategy (CDA) to prevent system performance degradation.



(a) MPC: Batch dissimilarity scores compared to the first batch (B_0).



(b) MPCAWIN: Batch dissimilarity scores compared to the batches set as Current Concept (B_0 and B_{86}), α threshold evolution, noisy ds identified with SAD and drift ds detected with KSWIN.

Fig. 11. Concept Drift Detection analysis.

5.5. The concept drift detection strategy

A crucial aspect of concept drift management concerns the time between the actual occurrence of the drift and its detection from which the following adaptation phase arises, Fig. 10(b).

The timely detection of concept drift is critical for maintaining system performance. To this end, this section presents the experiments aimed to assess the effectiveness of MPCAWIN and highlight the contribution of its components.

Fig. 11(a) illustrates the analysis of ImprTotal with the Multidimensional PCA Comparator (MPC), showing batch dissimilarity scores and the impact of concept drift. Fig. 11(b), on the other hand, shows the overall operation of MPCAWIN, emphasizing the roles of the Statistical Anomaly Detector (SAD) and CDD_{mono} (KSWIN).

In Fig. 11(a), the dissimilarity scores $\gamma_{0,i}$ between first batch B_0 and B_i are displayed. The dissimilarity score, as reported in Section 4.1, is calculated as the angle between the vectorial representation of the compared batches, varying in the range [0;180]. The difference between the distribution of dissimilarity scores before and after the concept drift (vertical red dotted line) is evident. Except for a few noisy outliers, most batches belonging to Impr2017, compared with B_0 (part of Impr2017), have low dissimilarity values, averaging just over 10 degrees (horizontal green dotted line). Conversely, batches belonging to Impr2018, compared to B_0 , show much higher dissimilarity values, averaging about 30 degrees (horizontal black dotted line). This proves that MPC can effectively measure dissimilarity based on the input data's feature distribution.

The primary challenge for MPCAWIN is to determine whether high dissimilarity values, which deviate significantly from the current distribution, are due to noise or indicate the presence of concept drift.

Fig. 11(b) shows dissimilarity values relative to the last detected concept. For the first 86 batches, comparisons refer to B_0 , as in Fig. 11(a). At batch 86 (red continuous line), drift is detected, and from that point on, comparisons refer to that batch. Dotted blue lines indicate values that are considered noisy because they exceeded the threshold α (continuous orange line) calculated on the distribution of the values collected in the sliding window A of the SAD component. As can be observed, dissimilarity scores of noisy batches are correctly identified as such, and no concept drift was detected due to their sporadic nature. When concept drift occurs (vertical red dotted line), dissimilarity scores are consistently recognized as noisy, leading to KSWIN detecting a change in the statistical distribution.

As shown in Fig. 10(b) and summarized in Table 2 (Delay), HOIDS evaluates 5 batches of data with the initial model before adapting, after the drift occurrence.

To understand more in detail how MPCAWIN performs, we can compare the dissimilarity values obtained with MPC alone and with MPCAWIN.

Using MPCAWIN (Fig. 11(b)), the average dissimilarity angle values for Impr2018 drop to about 23 degrees (black dotted line after drift) compared to 30 degrees with MPC alone, which is remarkable. However, the new distribution of dissimilarity values for Impr2018 has a higher variance than the distribution of Impr2017 values referred to B_0 . The A sliding window of SAD captures this new characteristic of the Impr2018 dissimilarity value distribution.

The threshold α , calculated as the mean plus two standard deviations of values contained in A, ensures that those higher values are not considered noisy, preventing false concept drift detection. In addition, the sliding window mechanism allows α to be calculated from the latest values in A, thus, as the mean and variance of the dissimilarity values decrease, the threshold is lowered and some batches are considered noisy from batch 140 onwards.

Setting a static threshold, as proposed in Shao et al. (2014), is challenging. The choice could be based on accepting a maximum dissimilarity difference to avoid performance degradation. However, concept drift not only increases dissimilarity between batches before and after

Table 3

Sensitivity analysis of MPCAWIN hyperparameters (c, m, bs) in terms of Precision of concept drift detection ($Prec_{cdd}$) and concept drift detection delay in terms of number of batches related to the new concept evaluated before the correct detection of drift (Delay). The values of the parameters used for MPCAWIN are shown in bold.

c	$Prec_{cdd}$	Delay	m	$Prec_{cdd}$	Delay	bs	$Prec_{cdd}$	Delay
0	–	–	3	–	–	500	–	–
1	0/1	–	5	1/1	6	1000	0/1	–
2-6	1/1	5	10	1/1	5	2000	1/1	5
7	–	–	15	1/1	7	3000	1/1	5
8	–	–	20	1/1	9	4000	1/1	7

the drift but also changes the dissimilarity value trend between batches belonging to the same concept. For example, the statistical distribution of dissimilarity values on Impr2017 is completely different from that on Impr2018. Therefore, a static threshold that initially worked may no longer be suitable after concept drift is detected.

This problem is mitigated by using a sliding window to estimate the distribution of dissimilarity values and calculate the threshold. In such scenario, however, it is very important not to include noisy values in the sliding window A. Without this precaution, a high α value just before the concept drift would prevent the detection of dissimilarity values resulting from the drift.

Conversely, in the worst case, an inaccurate distribution estimate and the exclusion of values that are part of the distribution (due to a low threshold) can lead a false detection of concept drift and unnecessary adaptation phases. To avoid this, after setting the new concept, MPCAWIN resets the sliding window A and re-estimates the dissimilarity value distribution. The experiments demonstrate the correct functioning of the three components of MPCAWIN and the advantages of their integration into a multidimensional Data Distribution-Based concept drift detector.

It is worth noticing that the correct detection and the time from drift occurrence to detection depends on the key parameters of MPCAWIN and KSWIN. For KSWIN, the sensitivity hyperparameter is set to the maximum value recommended by the authors, making the detector more reactive. This choice is motivated by the fact that only two discrete values (0 or 1), rather than a range of real values, are used. In addition, KSWIN requires specifying the size of the statistic window ss , which determines the amount of recent data to be compared with the historical data ws , potentially representing a “new concept”. Choosing this value involves a trade-off between noise resistance and detection speed.

In this case as well, the ratio between ss and ws recommended by the authors is maintained by adjusting the window sizes to the number of batches to be analyzed ($ss = 5$, $ws = 20$).

Regarding the MPCAWIN parameters, a sensitivity analysis is performed on the batch size bs , the size m of the sliding window A, and the constant c used to compute the threshold α , in order to evaluate their impact on the Precision of concept drift detection ($Prec_{cdd}$) and on drift detection delay. The results of this analysis are summarized in Table 3, and Fig. 12 illustrates representative cases from the sensitivity analysis. Together, these visualizations make the sensitivity analysis more interpretable and clarify the role of each parameter in the overall detection mechanism.

Regarding the parameter m , values that are too low (such as $m = 3$) are not sufficiently representative of the distribution for SAD: as shown in Fig. 12(a), the threshold α is estimated from too few values, causing SAD to classify the majority of dissimilarity scores as anomalous even before any drift occurs. This ensures that these outliers are not included in the recalculation of α that consequently remains persistently low. This saturates the binary stream fed to CDD_{mono} with 1 s, leaving no detectable change after the drift and ultimately leading to missed detections.



Fig. 12. Representative cases from the sensitivity analysis of MPCAWIN parameters, illustrating the behavior of the system for values of m , c , and bs outside the recommended range.

Values of m greater than 5 are more robust and lead to correct detections: as shown in Fig. 12(b), larger values yield a more reliable threshold estimate, allowing SAD to generate a sustained stream of anomalous flags specifically after the drift occurs, at the cost of an increasing detection delay. It is worth noting that, after a drift is detected, the window A is emptied and the computation of α must wait until m new samples are collected, introducing a necessary trade-off between stability and detection speed.

Excessively low values of c lead to thresholds that are too permissive, causing SAD to overreact to noise. More specifically, as shown in Fig. 12(c), when $c = 0$, similarly to the case $m = 3$ (Fig. 12(a)), virtually all dissimilarity scores are classified as anomalous by SAD. As a result, the binary stream fed to CDD_{mono} consists almost entirely of 1s from the

very beginning, leaving no detectable change in distribution after the drift occurs and preventing any detection. For intermediate low values such as $c = 1$, the threshold is slightly less permissive but nevertheless causes false detections before the drift takes place.

Conversely, excessively high values of c (Fig. 12(d)) produce a threshold so conservative that no dissimilarity scores are flagged as anomalous, neither the noisy values before the drift nor the genuinely elevated ones following it, resulting in a complete failure to detect the drift. Values of c between 2 and 6 appear adequate, triggering a single correct detection with a delay of 5 batches (Fig. 11(b)).

The parameter bs is subject to the same trade-off between stability and detection speed. If bs is too small, the resulting distribution of dissimilarity values becomes too noisy: as shown in Fig. 12(e), a batch

Table 4

Comparison of state-of-the-art mono-dimensional concept drift detectors when embedded in MPCAWIN and when used in standalone error-rate mode, in terms of concept drift detection precision ($Prec_{cdd}$) and detection delay (number of batches of the new concept evaluated before the correct detection of drift).

CDD_{mono}	MPCAWIN		Error-rate	
	$Prec_{cdd}$	Delay	$Prec_{cdd}$	Delay
KSWIN	1/1	5	0/1	–
ADWIN	1/1	60	1/11	1
HDDM _A	1/1	4	1/1	5
HDDM _W	1/1	12	0/1	–
PageHinkley	1/1	6	1/1	3

size of $bs = 1000$ produces unstable dissimilarity estimates that cause a false detection well before the actual drift occurs.

Values of $bs = 2000$ or higher tend to be sufficiently representative: as shown in Fig. 12(f), $bs = 4000$ yields correct detection, though at the cost of increased delay due to the larger number of records processed per batch. It is important to note that, although the batch delay remains constant, increasing bs also increases the total number of records processed between the drift and its detection.

To clarify the design rationale and advantages of MPCAWIN with respect to the state of the art, two experiments are presented: a direct comparison between MPCAWIN and several mono-dimensional error-rate detectors, including ADWIN, HDDM_A, HDDM_W, and Page-Hinkley; and an ablation study in which each of the above detectors replaces KSWIN as the CDD_{mono} component inside MPCAWIN.

In these experiments, the default hyperparameter configurations were used for all drift detectors. The results of this two-part analysis are summarized in Table 4. A particularly compelling result is that all detectors, regardless of their standalone performance, achieve perfect detection precision (1/1) when embedded in MPCAWIN. In this regard, KSWIN proves to be one of the best detectors, alongside HDDM_A. This confirms that the contribution of the MPC and SAD layers is substantial and largely independent of the specific detector used as the final hypothesis test, highlighting the robustness and modularity of the proposed architecture.

When used in error-rate mode, the same detectors suffer considerably from noise sensitivity in the IDS context, as shown in Table 4. The results show, for example, that ADWIN triggers 10 false positives out of 11 detections, while KSWIN and HDDM_W fail to detect the drift entirely. MPCAWIN, by contrast, achieves fully precise detection across all evaluated scenarios, confirming the effectiveness of its multi-layer architecture in filtering spurious deviations before they reach the hypothesis test stage.

It is also worth noting that a direct comparison between MPCAWIN and error-rate detectors is inherently asymmetric, since the latter require timely access to ground-truth labels to monitor classifier performance, while MPCAWIN operates entirely on the statistical distribution of input features without any labeling cost during the detection phase.

5.6. Evaluation of the active learning module

The AL module reduces the labeling burden on domain experts by limiting manual annotation to the samples for which the model exhibits the greatest uncertainty. More formally, the labeling cost is defined as:

$$L_c = \frac{|D_{labeled}|}{|W|} \cdot 100. \quad (12)$$

where $D_{labeled}$, in accordance with Alg. 1, contains the subset of records in W , selected through the AL module, that must be labeled manually.

It is worth clarifying the distinction between two related but different quantities reported in the paper. The GT% column of Table 2

Table 5

Classification performance achieved by HOIDS after the adaptation phase, comparing IAE against three standard active learning strategies at equal labeling cost.

AL	Acc.	Prec.	Recall	F1	FPR	FNR
IAE	0.998	0.980	0.954	0.967	0.0005	0.046
Uncertain	0.997	0.954	0.954	0.954	0.0013	0.046
Entropy	0.997	0.949	0.956	0.952	0.0015	0.044
Margin	0.995	0.902	0.955	0.928	0.0029	0.045

represents the fraction of records in the entire dataset that require manual annotation, and reflects the overall annotation burden across the full evaluation run. The labeling cost L_c represents the percentage of records in the retraining window W that must be submitted for expert review at each adaptation phase. The two quantities are proportional but carry different interpretations, and L_c is the more informative one for evaluating the active learning module in isolation.

In order to evaluate the impact of different approaches this sections compares the performance of HOIDS using Iterative AutoEncoder (IAE) approach with two different versions, named *HOIDS_NAE* and *HOIDS_BL*, which use the Naive AutoEncoder (NAE) or not include the Active Learning module at all, respectively. *HOIDS_BL*, considered as a baseline, involves human intervention to label data after each drift detection, before the adaptation phase. This comparison constitutes an ablation study on the AL module: *HOIDS_BL*, which excludes the AL module entirely and requires full manual labeling, serves as the baseline; *HOIDS_NAE* replaces IAE with the simpler Naive AutoEncoder; and HOIDS uses the full IAE approach. The three variants share the same MPCAWIN and XGBoost components, isolating the contribution of the AL module and its iterative algorithm.

As shown by the confusion matrix in Fig. 13(a), the *HOIDS_NAE* approach manages to recognize half of the values as benign without generating misclassifications. This result aligns with the decision to set the threshold τ as the median of the reconstruction values of the W retraining window data, halving the labeling effort for the domain expert compared to *HOIDS_BL*.

Fig. 13(b) shows how the IAE approach increasingly refines its results with the various iterations. The initial confusion matrix is the same of NAE (Fig. 13(a)). The subsequent confusion matrices show the number of additional records recognized as benign in each iteration.

The IAE approach converts the AutoEncoder from an unsupervised version with a conservative threshold to a semi-supervised version. The confusion matrix in the lower right shows the final result of the Iterative approach. IAE terminates early as no new records are detected as benign at the fifth iteration and recognized approximately 22% more benign records than the NAE without any misclassifications. As shown in Table 2, compared to state-of-the-art systems such as AutoML and AXGB and compared to *HOIDS_BL*, which require 100% of the retraining window to be labeled, HOIDS with IAE reduces this labeling cost to 39.15%, corresponding to a reduction of approximately 60% in labeling effort per adaptation phase. Over the full evaluation run, this translates to a GT% of 0.92% compared to 100% for the competing systems, while achieving a comparable or superior F1-score of 0.961.

In order to further assess the benefits provided to HOIDS by the proposed AL module, a direct comparison between IAE and three widely used active learning strategies (uncertainty sampling, entropy sampling, and margin sampling) (Settles, 2009) is conducted. As discussed in Section 4.2, these methods take as input the machine learning model used to evaluate the records and a query budget value. The output is a list of the most relevant records, as determined by the metric used, in a quantity sufficient to satisfy the query budget. The comparison is conducted at equal query budget or, in other words, labeling cost, with all methods selecting the same fraction of records for manual annotation (39.15%, as determined by IAE). Table 5 reports the classification performance achieved by HOIDS after the adaptation

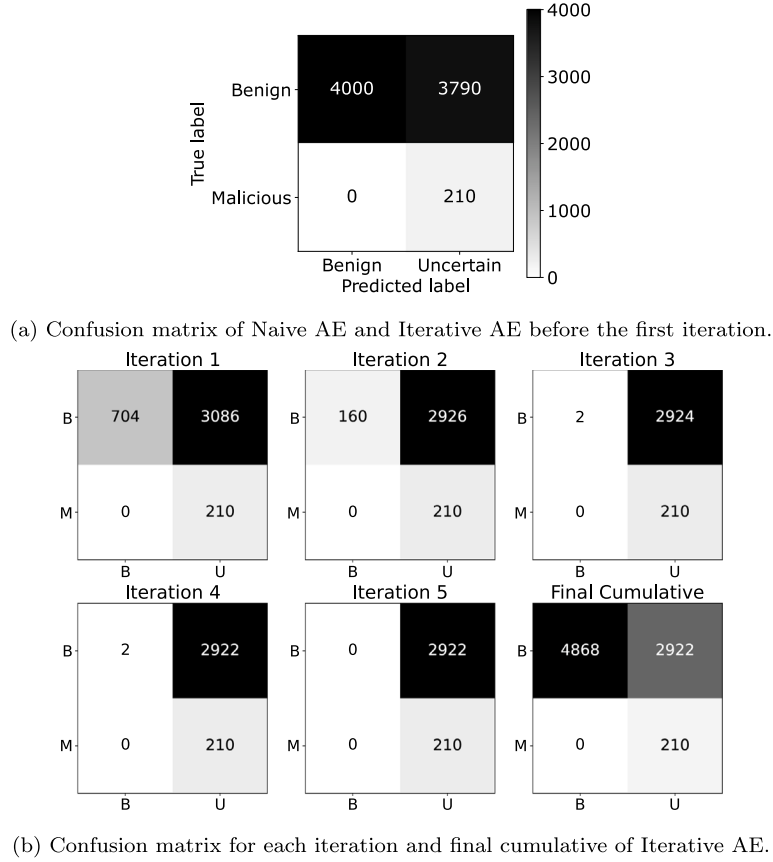


Fig. 13. Active Learning Module (AL) evaluation in terms of number of records properly self-labeled using the Naive AE approach (Fig. 13(a)) and the Iterative AE approach (Fig. 13(b)).

Table 6

Ablation study on the AutoEncoder architecture: IAE performance across different architectural configurations in terms of benign/uncertain classification accuracy and labeling cost.

Type	Conf.	Acc.	FPR	FNR	Lc (%)
<i>avg±std</i>	–	0.639 ± 0.033	0.369 ± 0.036	0.004 ± 0.009	38.44 ± 3.67
<i>IAE</i>	[512, 256, 64]	0.630	0.379	0.0	39.15
<i>best</i>	[1024, 256, 32]	0.660	0.348	0.0	36.56
<i>worst</i>	[1024, 256, 64, 32]	0.570	0.441	0.0	45.58

phase, comparing IAE against the three AL baselines at equal labeling cost.

In the IAE case, as shown in Alg. 1, RD consists of the data auto-labeled as benign by the proposed AL module and the remaining records labeled manually ($D_{labeled}$). Alternatively, for classical AL methods, RD consists exclusively of the records selected for manual labeling.

IAE achieves an F1-score of 0.967 against 0.954, 0.952, and 0.928 for the competing methods, and a FPR of 0.0005, which is between 3 and 6 times lower than the alternatives. In a security-critical context such as intrusion detection, this reduction in false positives is particularly significant.

This discrepancy in performance can be attributed to the enhanced representativeness of the retraining window RD achieved through IAE in comparison to conventional active learning methods. Indeed, for the same labeling cost, IAE auto-labeled more than half of the data without any misclassification errors, thereby increasing the number of benign records available for retraining and consequently reducing the number of false positives. These results highlight the relationship between the number of correctly labeled records available for the adaptation phase and the resulting system detection performance.

To better quantify the trade-off between labeling cost and detection performance, a budget-constrained analysis is conducted. The maximum sustainable labeling cost L_c^{max} is varied below the natural operating point of IAE ($L_c = 39.15\%$). Three approaches for handling the residual uncertain samples when $L_c^{max} < L_c$ are implemented and compared: the first submits only the most uncertain records for labeling and discards the rest; the second and third automatically label the residual uncertain samples as benign or malicious, respectively. The results, in terms of F1-score of HOIDS after adaptation, shown in Fig. 14, reveal that the discard approach is remarkably robust across the full range of budget constraints, maintaining an F1-score above 0.9 even at L_c as low as 5%, and reaching 0.97 at the natural IAE operating point. By contrast, automatically labeling residual uncertain samples as malicious or benign leads to severe performance degradation at low budgets, confirming that propagating incorrectly labeled samples (FN and FP) into the training set is significantly more harmful than simply reducing the size of the retraining set.

This final consideration highlights the importance of the robustness of the proposed AL module and its IAE version, as well as their ability to identify and automatically label benign records while avoiding misclassifications.

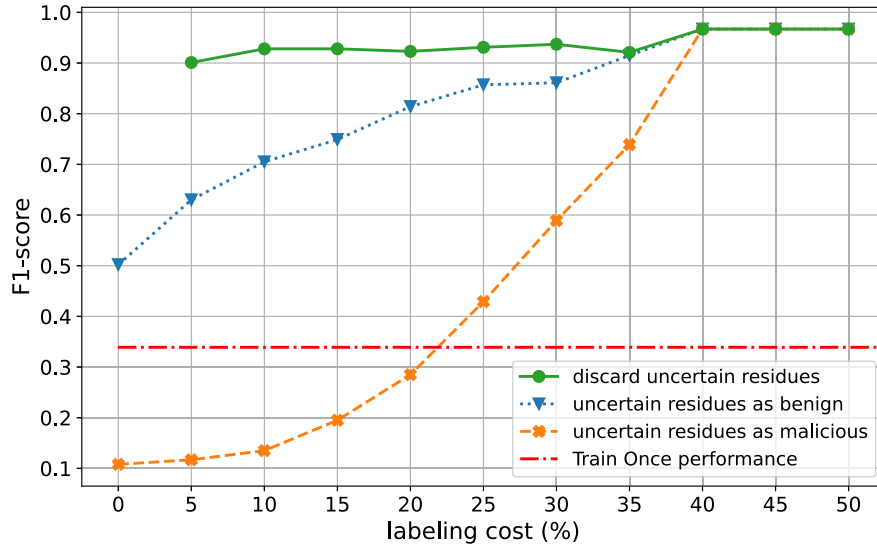


Fig. 14. Trade-off between labeling cost, false negative and false positive risk propagation into the retraining set, and HOIDS detection performance after adaptation. Three approaches are compared when the maximum labeling cost is lower than that required by IAE: labeling only the most uncertain records and discarding the rest; automatically labeling the remaining uncertain samples as benign or malicious, respectively.

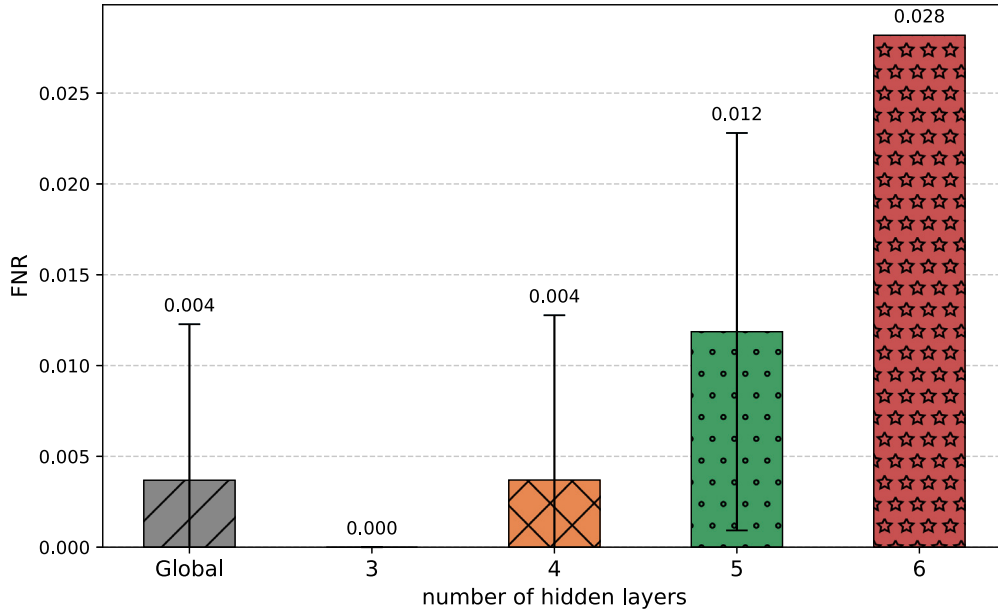


Fig. 15. Comparison of IAE performance varying the number of hidden layers in the AutoEncoder architecture in terms of mean and standard deviation of FNR.

To further verify the effectiveness of the proposed AL module when varying the AutoEncoder's architecture, an ablation study is conducted. The number of hidden layers was varied from 3 to 6, and their sizes from 1024 to 32, with the constraint that the size of the encoder's hidden layers must decrease strictly and that the decoder's layers must be symmetrical to the encoder.

The results in terms of benign/uncertain classification performance and L_c using IAE approach are reported in Table 6.

As shown, the standard configuration [512, 256, 64] achieves an FNR of 0.0 and a labeling cost of 39.15%, which is consistent with the average across all tested configurations ($\text{FNR} = 0.004 \pm 0.009$, $L_c = 38.44\% \pm 3.67\%$). The best-performing configuration [1024, 256, 32] yields a slightly lower labeling cost (36.56%) while maintaining $\text{FNR} = 0.0$, while the worst configuration [1024, 256, 64, 32] increases the labeling cost to 45.58%, still with $\text{FNR} = 0.0$.

It should be noted that the best and worst configurations are selected from among those with a zero FNR, which account for the majority of the total, approximately 81%. However, some configurations have an FNR other than zero, and even if they have better Acc. or L_c values than the worst configuration, they should be avoided due to the impact of FN on batch retraining, as discussed earlier.

Related to this, Fig. 15 shows the mean and standard deviation of the FNR, grouped by the number of hidden layers in the AutoEncoder. Notably, increasing the number of layers beyond 4 begins to degrade FNR, confirming that the standard 3-layer configuration represents a good trade-off between compactness and robustness. These results confirm that the robustness of IAE stems from the iterative algorithm and its stopping condition rather than from the specific AE architectural choice, and that the default configuration is a reasonable and well-supported selection.

Table 7

Strengths and weaknesses of compared systems in terms of performance in static environment (Perf. static env.), performance in open environment (Perf. open env.), labeling cost (Lab. cost), False Positive Rate of concept drift detection (C.d.d. FPR), and delay in concept drift detection (C.d.d. Delay). A five-level ordinal scale is adopted, comprising: Very High (VH), High (H), Medium (M), Low (L), and Very Low (VL). For performance metrics, higher ratings indicate better outcomes; for all other properties, lower ratings are preferable.

Approach	System	Perf. static env.	Perf. open env.	Lab. cost	C.d.d. FPR	C.d.d. Delay
Static	LCCDE	VH	VL	–	–	–
Periodic Retrain	LCCDE	VH	VH	VH	–	–
Detect and Retrain	AutoML	VH	VH	VH	H	L
Periodic Incr./Rep.in Ens.	AXGB	VH	H	VH	VH	VL
Detect and Retrain	HOIDS	VH	VH	VL	VL	M

5.7. Performance comparison of different online systems

The experiments in this section compare the performance of online systems selected as representatives of the main state-of-the-art concept drift detection and adaptation strategies (Fig. 4), evaluating their impact not only on overall detection performance but also on labeling cost, false detection rate, and detection delay.

The baselines were selected to represent four distinct categories of approaches: LCCDE Train Once represents the classical static IDS approach with no drift awareness, and serves as a lower bound on performance in the presence of concept drift. LCCDE Periodic Retrain represents the upper bound of labeling cost, with full periodic annotation regardless of whether drift has occurred, and provides an upper bound on achievable performance under the same classifier architecture. AutoML represents a drift-aware *Detect and Retrain* strategy based on error-rate detection, requiring 100% ground-truth labels both during detection and adaptation. AXGB represents a drift-aware *Periodic Increment and Replace in Ensemble* strategy, also based on error-rate detection and full annotation.

This categorization allows the comparison to isolate the contribution of different design choices, namely the CDD strategy, the CDA strategy, and the labeling cost, rather than simply ranking systems by overall performance.

Table 2 provides a comprehensive comparison of all evaluated systems along the key dimensions that characterize drift-aware IDS design. The CDD Strategy column captures the drift detection approach used by each system; the GT% column quantifies the labeling cost as the fraction of records requiring manual annotation over the full evaluation run; the Delay column reports the number of batches evaluated under a drifted concept before detection is triggered; and the $Prec_{cdd}$ column reflects the robustness of the drift detection mechanism, with HOIDS achieving a perfect detection precision of 1/1, compared to 1/15 for AutoML and 1/24 for AXGB. The remaining columns report standard classification performance metrics.

The first system, LCDDE with a Train Once CDA strategy and without a CDD strategy, represents a classical static IDS approach. This system does not require further ground truth data labeling after the initial training phase, but suffers from performance degradation due to concept drift.

The second system, LCDDE with a Periodic Retrain CDA strategy and without a CDD strategy, achieves significantly better results than the first system, with an F1-score of 0.963. However, this strategy requires 100% ground truth data labels for periodic retraining. Comparing this system with HOIDS, it is clear that the only useful retraining phase from a performance standpoint is the one following the real concept drift.

Indeed, HOIDS shows comparable results, especially with the second highest F1 score (0.961) among the compared systems. This demonstrates the benefits of integrating Concept Drift Detection strategies into

the online approach and tying the adaptation phase directly to drift detection, resulting in significant savings in computing resources. The same strategy is also used by the other two compared systems.

AutoML achieves comparable performance to LCDDE Periodic Retrain and HOIDS, with an F1 score of 0.951. However, it also requires 100% ground truth data labels due to its error rate-based CDD strategy, which must repeatedly compare the system's predictions with ground truth. Moreover, such a CDD strategy produces a moderate amount of false detections, which results in a waste of computational resources and is sensitive to data noise.

While it is fast at detecting drift with a delay of only two batches, its overall performance is inferior to that of HOIDS, as indicated by the recall score of 0.911 versus 0.943 and the FNR score of 0.097 versus 0.060. This suggests that retraining at false drifts increased the False Negative Rate. Indeed, if retraining is performed on noisy batches that slightly impaired system performance by misleading the error-rate based Concept Drift Detection, it is possible that this negatively affects the decision boundary of the machine learning model, leading to the misclassification of some malicious records.

The last system, AXGB, has the same weaknesses as AutoML but with an advantage in drift adaptation speed. By implementing an incremental CDA strategy, AXGB begins retraining using the data that caused the drift immediately after it occurs. However, this does not improve performance compared to other systems, with an F1-score of 0.804, which is the lowest among the online systems compared. As discussed in Section 2, in addition to an incremental ensemble learning strategy, AXGB also uses a Detect and Replace strategy. Although the system uses the drift-involved data early for retraining, in weighted ensemble voting, classifiers trained on such data influence detection only after a longer delay, i.e., when almost all previous classifiers are discarded. HOIDS does not suffer from this problem because it discards the old model and uses a new one, fully trained on the data that caused the drift.

In conclusion, as shown in Table 2 and summarized in Table 7, HOIDS with Iterative AE approach achieves very high performance, with an F1-score of 0.961, lower only than LCDDE with Periodic Retrain, while maintaining a very low rate of ground truth data labels required, only 0.92%, compared to the 100% of the other systems.

This result is made possible by the hybrid multi-layer approach architecture and the effectiveness of the multidimensional Distributed-data based drift detection module MPCAWIN, which enabled concept drift detection without requiring ground truth data. Moreover, MPCAWIN proved to be more robust to noise than the CDD strategies of the other systems, detecting the drift only once when needed and avoiding unnecessary adaptation steps (CDA).

These positive results are offset only by a higher delay of adaptation compared to other systems. This delay exposed the system to concept drift for five batches before activating the adaptation strategy. However, as can be seen from the reported metrics, this delay is acceptable, as the overall system metrics are comparable or superior to those of other systems using faster CDD systems. The comparison between HOIDS variants confirms the effectiveness of the AL module: the unsupervised active learning approach automatically labels the majority of benign records, significantly reducing the labeling cost while preserving detection performance.

The modular architecture of HOIDS allows the labeling cost to be adjusted by enabling or disabling the AL module or by switching between NAE and IAE. As shown in the next section, this flexibility also translates into a tunable trade-off between labeling cost and computational overhead. The Detect and Retrain strategy, combined with the unsupervised drift detection provided by MPCAWIN, further contributes to the stability of these results by limiting adaptation phases to genuine drift events, thereby avoiding the performance degradation that unnecessary retraining can introduce in Periodic and Periodic Increment strategies.

Table 8

Hardware and software configuration of the experimental server.

Hardware
16 cores CPU: Intel(R) Xeon(R) Gold 6326 CPU @ 2.90 GHz
RAM: 128 GB
GPU: NVIDIA A100 (vGPU GRID) - 20 GB VRAM
Software
OS: Ubuntu 24.04.2
Runtime: Python 3.9

Table 9

Computational overhead of the compared systems over the full evaluation run.

System	Comp. over.
<i>Train Once</i>	0.137 s
<i>Periodic Retrain</i>	5.9 s
<i>HOIDS_BL</i>	1.1 s
<i>HOIDS_NAE</i>	15.7 s
<i>HOIDS</i>	31.1 s

5.8. HOIDS real-time feasibility

This section analyzes the real-time feasibility of HOIDS by discussing the computational overhead of each individual component and of the system as a whole, the memory footprint during the experimental runs, and the hardware and software configuration used for all experiments, reported in Table 8.

The computational overhead of each individual component of HOIDS is summarized in Table 10. The per-component overhead shows that MPCAWIN introduces a negligible processing cost of 0.0049 s per batch, and XGBoost inference requires 0.0066 s per batch on average. The dominant cost is the IAE retraining step. Since HOIDS triggers retraining only upon drift detection, this cost is incurred once per drift event rather than continuously.

Regarding memory consumption, the total memory footprint of HOIDS during the experimental runs amounts to approximately 4% of the available RAM on the experimental server (Table 8).

Over the full evaluation run, the total computational overhead of HOIDS amounts to 31.1 s, compared to 5.9 s for the Periodic Retrain approach, as reported in Table 9. We consider this difference acceptable for a centralized deployment scenario, where the system can run on a dedicated server without stringent hardware constraints. It is also worth noting that, in a real deployment, the dominant time cost is not computational but rather the expert annotation phase. Since HOIDS reduces the fraction of records in the entire dataset that require manual annotation to 0.92%, the practical turnaround time between drift detection and successful adaptation is considerably lower than in systems that require full manual annotation, regardless of the computational overhead.

To further contextualize these results, consider a realistic deployment scenario in which HOIDS monitors the traffic of a corporate subnetwork. Network traffic is inherently variable and burst-driven, and the available bandwidth is rarely fully utilized. Even in a worst-case scenario, assuming half the maximum bandwidth of a 1 Gbps link, approximately 120,000 packets per second would be exchanged. Assuming an average of 100 packets per flow record, this corresponds to roughly 1200 records per second, meaning that a batch of $bs = 2000$ records would be filled in approximately 2 s. MPCAWIN and XGBoost process each such batch in under 0.012 s in total, introducing no bottleneck in the detection pipeline under normal operating conditions. In the event of drift, with a detection delay of 5 batches and an adaptation overhead of 31.1 s, the system evaluates approximately 20 batches under the drifted concept before successfully adapting. In higher-speed scenarios, such as a 10 Gbps link at half capacity, the batch fill time drops to approximately 0.33 s, and the system would evaluate around

Table 10

Computational overhead of each individual component of HOIDS.

HOIDS component	Avg batch processing time
MPCAWIN	0.0049 s
IAE	30 s
XGBoost	Inference: 0.0066 s; Training: 0.137 s

Table 11

Performance comparison of HOIDS and ENIDrift on the RWDIDS-12 IoT dataset in terms of Precision (Prec.), Recall (Rec.), F1-score (F1), and percentage of records requiring manual annotation (GT%).

System	Prec.	Rec.	F1-score	GT%
<i>ENIDrift-PCA</i>	0.690	0.919	0.789	100%
<i>ENIDrift-AE</i>	0.701	0.944	0.804	100%
<i>Train Once</i>	0.889	0.824	0.855	0%
<i>HOIDS_BL</i>				7.5%
<i>HOIDS_NAE</i>	0.908	0.824	0.864	3.75%
<i>HOIDS</i>				3.2%

100 batches under the drifted concept before adapting. While this represents a more significant exposure, we note that adaptive batch sizing could further mitigate this trade-off by dynamically adjusting bs based on observed traffic intensity, and we identify this as a direction for future work.

Furthermore, as shown in Table 3, the sensitivity analysis on bs confirms that batch sizes below 2000 records produce unreliable detections, while values of 2000 or above yield consistent and correct drift detection. The chosen value of $bs = 2000$ therefore represents the minimum batch size that guarantees reliable detection while minimizing the number of records evaluated under a drifted concept.

5.9. HOIDS generalizability

To evaluate the generalizability of HOIDS beyond enterprise network traffic, the system has been assessed on RWDIDS-12, an IoT network traffic dataset featuring concept drift, proposed by the authors of ENIDrift (Wang, 2022). This dataset contains approximately 400,000 records with 200 features and a malicious record ratio of approximately 23%. For these experiments, the batch size was set to $bs = 5000$ and the retraining window to $W = 30000$ records, while all other hyperparameters were kept as described in Section 5.2.

The results are reported in Table 11, alongside the performance reported by the ENIDrift authors for their own system on the same dataset. HOIDS achieves a Precision of 0.908 and an F1-score of 0.864, outperforming both ENIDrift-PCA (F1 = 0.789) and ENIDrift-AE (F1 = 0.804). ENIDrift achieves higher Recall, reflecting its design orientation towards minimizing false negatives at the cost of a lower Precision.

MPCAWIN correctly detects a drift event in this dataset; however, unlike the ImprTotal dataset, RWDIDS-12 does not provide explicit annotations of the drift occurrence time, making it impossible to measure the detection delay relative to the actual drift. The improvement in Precision and F1-score compared to the Train Once baseline nonetheless suggests that the adaptation successfully prevented the misclassification of benign records following the drift. Furthermore, IAE successfully auto-labels a significant fraction of benign records, reducing the labeling cost from 7.5% (HOIDS_BL) to 3.2% (HOIDS), confirming its effectiveness in a domain where benign traffic constitutes the majority of records.

These results confirm that MPCAWIN, IAE, and XGBoost are adaptable to contexts sharing the key characteristics of the online IDS setting, namely high-dimensional records and a dominant benign class.

To provide a further cross-domain perspective, ENIDrift was also evaluated on ImprTotal. The system achieved an Accuracy of 0.925 but an F1-score below 0.1, a result consistent with the difficulty reported by the ENIDrift authors themselves on CIC-IDS-2017, where they report

an F1-score of approximately 0.491. On ImprTotal, this difficulty was further compounded by the concept drift, which was not correctly managed, leading to a substantial increase in both false positives and false negatives following the drift transition. This cross-evaluation highlights a key difference in generalizability: while ENIDrift is specifically optimized for the IoT domain, HOIDS achieves competitive or superior performance across both enterprise and IoT network traffic scenarios, demonstrating broader applicability without domain-specific tuning.

6. Conclusion and future work

This work highlights open challenges for modern Intrusion Detection Systems (IDS) based on machine learning models. The first challenge concerns the poor generalization of the techniques most commonly used by static systems, trained on a single dataset, with consequent degradation of inter-dataset performance. We address this problem by transforming static approaches into an online approach that explicitly handles concept drift.

The second challenge is to minimize the number of ground truth data labels required by the concept drift detection and adaptation strategy. This minimization is crucial to reduce the labeling burden on domain experts and make the approach more practical for the IDS domain.

To overcome these challenges, we propose HOIDS, a hybrid online IDS that uses a multidimensional concept drift detection method based on the statistical distribution of input data features. We also enriched the commonly used Detect and Retrain concept drift adaptation strategy by introducing an unsupervised AutoEncoder-based active learning module to distinguish between benign and uncertain traffic and reducing the cost of labeling.

An extensive experimental evaluation has been performed on two datasets obtained from real network traffic: CIC-IDS-2017 and CSE-CIC-IDS-2018, which are currently considered primary benchmarking datasets for NIDS. Such an evaluation allowed to assess the performance of MPCAWIN, demonstrating its robustness and noise resistance as a concept drift detection algorithm. The performance comparison of HOIDS with static state-of-the-art methods, highlighted the negative impact of concept drift on these techniques and proved how HOIDS effectively mitigates this issue. Furthermore, the experimental comparisons with three online systems shows that HOIDS achieves comparable or superior performance with an F1-score of 0.961, while significantly reducing the labeling cost in terms of number of ground truth data labels required. A promising direction for future work is the exploration of kernel PCA as a more expressive alternative to the current linear projection, with the aim of better capturing non-linear drift patterns with MPCAWIN. This extension would however require carefully assessing the trade-off between improved robustness and the substantially higher computational cost that kernel PCA entails compared to standard PCA. We also plan to investigate adaptive window sizing and batch size adjustment to better handle periods of varying traffic intensity, thereby enabling more timely detection of and adaptation to drift.

Additionally, we plan to equip HOIDS with a mechanism to handle recurring concept drift, allowing the system to reuse previously learned models when a known concept reappears and avoiding unnecessary retraining. A further direction concerns the evaluation of HOIDS under adversarial conditions: an attacker aware of the drift detection mechanism could deliberately manipulate the input distribution to evade detection or to trigger unnecessary adaptation phases, for instance by crafting traffic patterns designed to confuse MPCAWIN or to inflate the labeling cost through spurious uncertain samples. Assessing the robustness of HOIDS against such threat models represents an important open challenge for its deployment in real-world adversarial environments.

A further direction for future work is the evaluation of HOIDS on scenarios involving multiple drift events of varying types and intensities. However, the datasets commonly used in the literature to

test concept drift management are either artificially generated or insufficiently challenging from a classification standpoint, as they lack the high-dimensional feature space and realistic class imbalance that characterize network intrusion detection datasets.

Therefore, there is still a need to study and collect network traffic datasets with the positive characteristics of CIC-IDS2017 and CSE-CIC-IDS2018, but coming from real and realistic long-duration analyses to capture the real occurrence of different concept drifts and the appropriate distribution of benign and malicious network traffic. Network datasets with the characteristics mentioned above would allow us to expand the study through a more rigorous analysis of the trade-off between the computational cost of the training phase, the labeling cost, and real-time operational constraints. Such an analysis would require a model that accounts for a delay in sample labeling proportional to the volume of labels required, as well as an evaluation of records that takes into account the temporal relationship between the adaptation time and the actual arrival of data streams. This would allow for the formulation of relevant considerations and the evaluation of automatic mechanisms for adaptively requesting samples to be labeled, and thus the labeling cost to be sustained, based on both the magnitude of the drift detected by MPCAWIN and the observed traffic volume and thus the number of records to be analyzed within a given time interval.

From a design perspective, the components of HOIDS are not specific to enterprise network traffic. MPCAWIN operates on the statistical distribution of input features and is applicable to any high-dimensional data stream where distribution shifts are expected over time, regardless of the domain. The proposed AL module is particularly well-suited to settings where the benign class dominates, which is a common characteristic of IoT and industrial control system monitoring, where normal operation constitutes the vast majority of observed traffic. The XGBoost classifier supports both binary and multi-class classification, making it applicable to a broad range of detection tasks. An initial validation of this generalizability is provided in Section 5.9, where HOIDS is evaluated on the RWDIDS-12 IoT dataset. We plan to further extend this evaluation to other IoT datasets such as CIC-IoT 2023 (Neto et al., 2023), industrial network datasets and UAV networks such as UAV-NIDD (Hadi et al., 2025).

CRedit authorship contribution statement

Alessandra De Paola: Writing – review & editing, Visualization, Supervision, Formal analysis, Conceptualization. **Salvatore Drago:** Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Pierluca Ferraro:** Writing – original draft, Visualization, Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Giuseppe Lo Re:** Writing – review & editing, Supervision, Resources, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the AMELIS project, within the project FAIR (PE0000013), and by the ADELE project, within the project SERICS (PE00000014), both under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

Data availability

Data will be made available on request.

References

- Abuadlla, Yousef, Kvascev, Goran, Gajin, Slavko, Jovanovic, Zoran, 2014. Flow-based anomaly intrusion detection system using two neural network stages. *Comput. Sci. Inf. Syst.* 11 (2), 601–622.
- Agate, Vincenzo, De Paola, Alessandra, Drago, Salvatore, Ferraro, Pierluca, Lo Re, Giuseppe, 2024. Enhancing IoT network security with concept drift-aware unsupervised threat detection. In: 2024 IEEE Symposium on Computers and Communications. ISCC, IEEE, pp. 1–6.
- Agate, Vincenzo, Drago, Salvatore, Ferraro, Pierluca, Lo Re, Giuseppe, 2022. Anomaly detection for reoccurring concept drift in smart environments. In: 2022 18th International Conference on Mobility, Sensing and Networking. MSN, IEEE, pp. 113–120.
- Baena-Garcia, Manuel, del Campo-Ávila, José, Fidalgo, Raul, Bifet, Albert, Gavalda, Ricard, Morales-Bueno, Rafael, 2006. Early drift detection method. In: Fourth International Workshop on Knowledge Discovery from Data Streams. vol. 6, Citeseer, pp. 77–86.
- Bayram, Barış, Köroğlu, Bilge, Gönen, Mehmet, 2020. Improving fraud detection and concept drift adaptation in credit card transactions using incremental gradient boosting trees. In: 2020 19th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 545–550.
- Bifet, Albert, Gavalda, Ricard, 2007. Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining. SIAM, pp. 443–448.
- Celik, Bilge, Vanschoren, Joaquin, 2021. Adaptation strategies for automated machine learning on evolving data. *IEEE Trans. Pattern Anal. Mach. Intell.* 43 (9), 3067–3078.
- Chen, Tianqi, Guestrin, Carlos, 2016. Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794.
- D'hooge, Laurens, Wauters, Tim, Volckaert, Bruno, De Turck, Filip, 2020. Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. *J. Inf. Secur. Appl.* 54, 102564.
- Ding, Zhiguo, Fei, Minrui, 2013. An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window. *IFAC Proc. Vol.* 46 (20), 12–17.
- Fogla, Prahlad, Sharif, Monirul Islam, Perdisci, Roberto, Kolesnikov, Oleg M, Lee, Wenke, 2006. Polymorphic blending attacks. In: USENIX Security Symposium. pp. 241–256.
- Frias-Blanco, Isvani, del Campo-Ávila, José, Ramos-Jimenez, Gonzalo, Morales-Bueno, Rafael, Ortiz-Diaz, Agustin, Caballero-Mota, Yailé, 2014. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Trans. Knowl. Data Eng.* 27 (3), 810–823.
- Gama, João, Žliobaitė, Indrė, Bifet, Albert, Pechenizkiy, Mykola, Bouchachia, Abdelhamid, 2014. A survey on concept drift adaptation. *ACM Comput. Surv. (CSUR)* 46 (4), 1–37.
- Hadi, Hassan Jalil, Cao, Yue, Khan, Muhammad Khurram, Ahmad, Naveed, Hu, Yulin, Fu, Chao, 2025. UAV-NIDD: A dynamic dataset for cybersecurity and intrusion detection in UAV networks. *IEEE Trans. Netw. Sci. Eng.* 12 (4), 2739–2757. <http://dx.doi.org/10.1109/TNSE.2025.3553442>.
- Hinder, Fabian, Vaquet, Valerie, Brinkrolf, Johannes, Artelt, André, Hammer, Barbara, 2022. Localization of concept drift: Identifying the drifting datapoints. In: 2022 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–9.
- Hinder, Fabian, Vaquet, Valerie, Hammer, Barbara, 2024. One or two things we know about concept drift—A survey on monitoring in evolving environments. Part A: Detecting concept drift. *Front. Artif. Intell.* 7, 1330257.
- Hurtado, Julio, Salvati, Dario, Semola, Rudy, Bosio, Mattia, Lomonaco, Vincenzo, 2023. Continual learning for predictive maintenance: Overview and challenges. *Intell. Syst. Appl.* 19, 200251.
- Jordaney, Roberto, Sharad, Kumar, Dash, Santanu K, Wang, Zhi, Papini, Davide, Nouretdinov, Iliia, Cavallaro, Lorenzo, 2017. Transcend: Detecting concept drift in malware classification models. In: 26th USENIX Security Symposium (USENIX Security 17). pp. 625–642.
- Lemaître, Guillaume, Nogueira, Fernando, Aridas, Christos K., 2017. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *J. Mach. Learn. Res.* 18 (17), 1–5, URL <http://jmlr.org/papers/v18/16-365.html>.
- Li, Wenjuan, Meng, Weizhi, Au, Man Ho, 2020. Enhancing collaborative intrusion detection via disagreement-based semi-supervised learning in IoT environments. *J. Netw. Comput. Appl.* 161, 102631.
- Liao, Hung-Jen, Lin, Chun-Hung Richard, Lin, Ying-Chih, Tung, Kuang-Yuan, 2013. Intrusion detection system: A comprehensive review. *J. Netw. Comput. Appl.* 36 (1), 16–24.
- Liu, Lisa, Engelen, Gints, Lynar, Timothy, Essam, Daryl, Joosen, Wouter, 2022. Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018. In: 2022 IEEE Conference on Communications and Network Security. CNS, IEEE, pp. 254–262.
- Lu, Jie, Liu, Anjin, Dong, Fan, Gu, Feng, Gama, Joao, Zhang, Guangquan, 2018. Learning under concept drift: A review. *IEEE Trans. Knowl. Data Eng.* 31 (12), 2346–2363.
- Marshall, Albert W., Olkin, Ingram, 1960. Multivariate chebyshev inequalities. *Ann. Math. Stat.* 1001–1014.
- Montiel, Jacob, Mitchell, Rory, Frank, Eibe, Pfahringer, Bernhard, Abdesslem, Talel, Bifet, Albert, 2020. Adaptive xgboost for evolving data streams. In: 2020 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.
- Neto, Euclides Carlos Pinto, Dadkhah, Sajjad, Ferreira, Raphael, Zohourian, Alireza, Lu, Rongxing, Ghorbani, Ali A., 2023. CICIOT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 23 (13), 5941.
- Page, Ewan S., 1954. Continuous inspection schemes. *Biometrika* 41 (1/2), 100–115.
- Perumal, Seethalakshmi, Sujatha, P Kola, Krishnan, Muralitharan, et al., 2025. Clusters in chaos: A deep unsupervised learning paradigm for network anomaly detection. *J. Netw. Comput. Appl.* 235, 104083.
- Raab, Christoph, Heusinger, Moritz, Schleif, Frank-Michael, 2020. Reactive soft prototype computing for concept drift streams. *Neurocomputing* 416, 340–351.
- Ramírez-Gallego, Sergio, Krawczyk, Bartosz, García, Salvador, Woźniak, Michał, Herrera, Francisco, 2017. A survey on data preprocessing for data stream mining: Current status and future directions. *Neurocomputing* 239, 39–57.
- Sagi, Omer, Rokach, Lior, 2018. Ensemble learning: A survey. *Wiley Interdiscip. Rev.: Data Min. Knowl. Discov.* 8 (4), e1249.
- Settles, Burr, 2009. Active learning literature survey.
- Shana, Tibebe Bekele, Kumari, Neetu, Agarwal, Mayank, Mondal, Samrat, Rathnayake, Upaka, 2025. Anomaly-based intrusion detection system based on SMOTE-IPF, Whale Optimization Algorithm, and ensemble learning. *Intell. Syst. Appl.* 200543.
- Shao, Junming, Ahmadi, Zahra, Kramer, Stefan, 2014. Prototype-based learning on concept-drifting data streams. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 412–421.
- Sharafaldin, Iman, Lashkari, Arash Habibi, Ghorbani, Ali A., et al., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSP* 1, 108–116.
- Tharwat, Alaa, Schenck, Wolfram, 2023. A survey on active learning: State-of-the-art, practical challenges and research directions. *Mathematics* 11 (4), 820.
- Togbe, Maurras Ulbricht, Chabchoub, Yousra, Boly, Aliou, Barry, Mariam, Chiky, Raja, Bahri, Maroua, 2021. Anomalies detection using isolation in concept-drifting data streams. *Computers* 10 (1), 13.
- Vo, Hoang V., Du, Hanh P., Nguyen, Hoa N., 2023. Ai-powered intrusion detection in large-scale traffic networks based on flow sensing strategy and parallel deep analysis. *J. Netw. Comput. Appl.* 220, 103735.
- Wahab, Omar Abdel, 2022. Intrusion detection in the IoT under data and concept drifts: Online deep learning approach. *IEEE Internet Things J.* 9 (20), 19706–19716.
- Wang, Xian, 2022. Enidrift: A fast and adaptive ensemble system for network intrusion detection under real-world drift. In: Proceedings of the 38th Annual Computer Security Applications Conference. pp. 785–798.
- Wold, Svante, Esbensen, Kim, Geladi, Paul, 1987. Principal component analysis. *Chemometr. Intell. Lab. Syst.* 2 (1–3), 37–52.
- Yang, Li, Shami, Abdallah, Stevens, Gary, De Russett, Stephen, 2022. LCCDE: A decision-based ensemble framework for intrusion detection in the internet of vehicles. In: GLOBECOM 2022-2022 IEEE Global Communications Conference. IEEE, pp. 3545–3550.
- Zhou, Yuyang, Cheng, Guang, Jiang, Shanjing, Dai, Mian, 2020. Building an efficient intrusion detection system based on feature selection and ensemble classifier. *Comput. Netw.* 174, 107247.